

Plotting an Estimation for the Boundary of $C'(1/6)$ Small Cancellation groups.

By

Shakespeare, Bryce

Submitted to the University of Bristol in accordance with
the requirements of the degree of MSc in Mathematical
Sciences in the Faculty of Science.

September 12, 2022

I declare that the work in this dissertation was carried out in accordance with the Regulations of the University of Bristol. The work is original, except where indicated by special reference in the text, and no part of the dissertation has been submitted for any other academic award. For all ideas taken from other sources (books, articles, internet), the source of the ideas is mentioned in the main text and fully referenced at the end of the report.

All material which is quoted essentially word-for-word from other sources is given in quotation marks and referenced.

Pictures and diagrams copied from the internet or other sources are labelled with a reference to the web page or book, article etc.

Any views expressed in the dissertation are those of the author.

SIGNED: Bryce Shakespeare.

DATED: September 12, 2022

Abstract

This project discusses the creation of a program aimed to plot an estimate of the boundary of hyperbolic groups. We begin with discussion of the theory of hyperbolic groups and the motivations behind estimating their boundaries. Small cancellation theory supplies a method to consistently construct hyperbolic groups, and acts as basis then for the following work. Having discussed the theory and motivation, we work towards developing our methods. Through the Knuth-Bendix algorithm, applied with the *kbmag* package in GAP, and MDS, applied via *sklearn* in Python, we construct our program. Discussing these alongside the capabilities of the program. Finalising with examples and applications.

Contents

1	Introduction	2
2	Hyperbolic Groups	4
2.1	Words, Generating Sets and Free Groups	4
2.2	Cayley Graphs	6
2.3	δ -Slim Triangles and Hyperbolicity	10
2.4	Gromov Boundaries	15
3	Small Cancellation Theory	20
3.1	Group Presentations	21
3.2	Small Cancellation	25
3.3	Dehn's Algorithm	27
4	Methods	29
4.1	The Knuth-Bendix Completion Algorithm	31
4.2	Word Generation	33
4.3	Dissimilarity Matrix	34
4.4	Multidimensional Scaling	35
5	Results and Experimentation	37
5.1	The Free Group	39
5.2	Input Parameters	40
5.3	Plotting an Estimation for the Boundary of $C'(1/6)$ Small Cancellation Groups	44
	Bibliography	47
A	GAP functions	50
B	Python functions	55

1 Introduction

Introduced and considerably developed by M. Gromov in a pivotal piece in 1987, hyperbolic groups aim to connect the properties of spaces with the algebraic properties of groups [25]. These ideas follow closely to that of combinatorial group theory and have lead an explosion in growth and impact in geometric group theory [25], becoming fundamental in the field. The inspirations follow from a line of existing results, one of these being the work of Dehn, solving the word problem by use of hyperbolic geometry and surface groups. Another expansion of Dehn's work was that of Greendlinger and Tartakovskii, leading to small cancellation theory [8], which is fundamental in this article due to its own connections to hyperbolic groups.

Given a subset of a group, $S \subset G$, we construct words in S as a concatenation of the elements of $S \cup S^{-1}$. If for all elements $g \in G$, we have a word in S that represents G then we call S a generating set of G . An equal definition being that S is a generating set if $\langle S \rangle$, the subgroup generated by S , is equal to G . This leads use to a definition of a metric on G regarding to S ; the word metric assigns length to an element $g \in G$, equal to the smallest number of elements in S needed to represent it. We can generate a Cayley graph on a group regarding this word metric, producing a proper geodesic metric space for any group with a finite generating set. This leads us to the notion of hyperbolicity.

Definition. A geodesic metric space, X , is called δ -hyperbolic, or Rips-hyperbolic, if all geodesic triangles of X are δ -slim.

Extended to groups, if the Cayley graph of a group is hyperbolic, then we say that group is *hyperbolic*, also referred to as *word-hyperbolic* or *Gromov-hyperbolic*.

Hyperbolic groups have an array of interesting properties and features. We concern ourselves with the Gromov boundary ∂X . The set of points at infinity of a space, the study of which is essential to the study of the overall space [25]. Defined by an equivalence class of infinite series, extending from a fixed point, formally:

Definition. Let X be a δ -hyperbolic geodesic metric space. Then

$$\partial X = \{[(x_i)] \mid (x_i) \text{ is a sequence that converges to infinity}\}$$

is the boundary of X .

In addition to inducing a rich, natural topology, the Gromov boundary admits a *visual metric*, which can be constructed as to emulate the Euclidean metric [10, Chapter 3], [25]. Capability to visualise this boundary would allow for further study of a groups properties and categorisation; the visual metric it bears marks a crucial step towards doing so.

In order to generate hyperbolic groups consistently and easily we turn to small cancellation theory. Dehn introduced the *word problem* following the formal definition and development of a groups presentation by Dyck [8]. The aim of solving which lead to an abundance of important theorems, and in turn the development of small cancellation theory [8]. Dehn's own work, and small cancellation theory, in turn acting as basis for Gromov's in hyperbolic groups [10, Chapter 1] .

A presentation of a group G , denoted $\langle S|R \rangle$, is a method of displaying or specifying a group. We take the free group $F(S)$ and $\langle\langle R \rangle\rangle$, the normal closure of R in $F(S)$. Then $G = F(S)/\langle\langle R \rangle\rangle$. S is the set of *generators* and R the set of *relators* [27, Chapter V]. Small cancellation aims to control and define the cancellation of groups, via conditions on the relators and *pieces*.

Definition. Let $G = \langle S|R \rangle$ with R *symmetrized*. If for any $r = uw_1 \in R$ where u is a *piece*, $\langle S|R \rangle$ satisfies the small cancellation condition $C'(\lambda)$ if

$$|u|_s < \lambda|r|_s.$$

A major result of small cancellation being Greendlinger's lemma.

Lemma. Greendlinger's lemma. *Let G presented by $\langle S|R \rangle$ satisfying $C'(1/6)$. For every non-trivial word w in S such that $w =_G 1$ there is a subword u of w and $r \in R$ such that u is also a subword of r and $|u|_s > (1 - 3\lambda)|r|_s$*

Through applications of Greendlings lemma, and further investigation upon the word problem, we can link small cancellation with hyperbolic groups.

Theorem. *Let G be a group with a presentation satisfying small cancellation condition $C'(1/6)$, then G is hyperbolic.*

This theory acting as a backbone for this article. Through easily determinable conditions on a presentation, one can consistently, easily, and quickly produce groups known to be hyperbolic. With this result in mind we now work towards developing a method to plot the boundary of such groups.

The Knuth-Bendix completion algorithm, if successful, produces a confluent, terminating rewriting system for a group given its presentation [23] [11]. This solves the word problem for the group, the conjugacy problem and makes way for many methods of analysing the group [2]. The *kbmag* package, developed by Derek Holt, applies this algorithm and gives many useful functions with a variety of applications. After successful application of the Knuth-Bendix algorithm on a group, we generate a set of words $\{w_1, w_2, \dots, w_n\}$ on a sphere in the Cayley graph of the group, sufficiently

far enough to represent the boundary. On this set of words we continue to generate a dissimilarity matrix D such that

$$D_{i,j} = e^{-\epsilon(w_i, w_j)_1}$$

for some $\epsilon > 0$, as to best mimic the visual metric. Our last step towards plotting the boundary is the translation of this matrix into Euclidean space and generating the points.

The method we chose to do such a task is *absolute multidimensional scaling*. Absolute MDS aims to produce a set of points $\{y_1, y_2, \dots, y_n\}$ in Euclidean space that best maintain the pairwise dissimilarities supplied in D within the new space [5]. This is done via minimisation of the *stress* function, defined as

$$\text{stress}_f\{w_1, w_2, \dots, w_n\} = \sqrt{\frac{\sum (f(D_{i,j}) - d(y_i, y_j))^2}{\sum d(y_i, y_j)^2}}.$$

The choice of ϵ producing D has heavy affect on the output from MDS following properties of the visual metric and it is recommended to start at $\epsilon = 0.25$, adjusting if necessary.

With the generation of Euclidean points, all that remains is to plot them. The functions developed have choices of 2 or 3 dimensions and can produce interactive plots if required. With functions developed alongside this article one can plot the estimate of the boundary of any supplied $C'(1/6)$ group (assuming Knuth-Bendix algorithm convergence) according to my method. This article also aims to help one understand the output and behaviours of such plots. A general idea of the shape of many hyperbolic groups Gromov boundary can be obtained with application of these functions.

2 Hyperbolic Groups

This section assumes knowledge of metric spaces covered in chapter 1 of [7], group theory in chapters 1 of both [30] and [29]. Some small graph theory will be used, covered in [12]. A rudimentary understanding of topology is certainly helpful but not necessary.

Hyperbolic groups were defined by and primarily developed by M. Gromov [17] in an incredibly influential article from 1987 with almost 4000 google scholar citations. This section aims to describe and define them, and construct a dialogue around why we are interested in their boundaries.

2.1 Words, Generating Sets and Free Groups

To begin with hyperbolic groups, we must start by introducing words and generating sets, then explore a few of their properties. These definitions follow predominantly from [22] and [15].

Definition 2.1. *Word.* Let G be a group and $S \subset G$ a subset of elements of G . A Word in S is a concatenation of elements of S and their inverses.

Example

Let $a, b, c \in G$. Then abc , $a^{-1}b^{-1}c^{-1}$, $a^{-1}aa^{-1}a$, $c^{-1}baaabb^{-1}c$ are words in $\{a, b, c\}$.

For a word $w = s_1s_2\dots s_n$ in S , a *subword* of w is a word $u = s_is_{i+1}\dots s_{i+m}$ such that $1 \leq i \leq n$ and $i + m \leq n$.

Any pair in a word where an element is adjacent to its inverse can be removed from the word, an operation known as reduction. The word resulting from doing as many reductions on a word as possible is called a *reduced word*. This gives way to the convention that the identity element of a group can be represented by the *empty word*, the word of length 0. Reduction also produces an obvious equivalence class on a set of words in S , whereby $w_1 \sim w_2$ if we can produce one from the other, by either adding or removing series of ss^{-1} and $s^{-1}s$ ($s \in S$).

We will also adopt the convention from here on that all words are reduced. For any w a word, by repeatedly removing all pairs ss^{-1} and $s^{-1}s$, we produce a reduced word equivalent to w . All equivalence classes of words will contain a unique reduced word, so this is not a problematic assumption. Out of convenience it is also convention that consecutive equal elements will be written with exponents, i.e $a^0 = e$, $aaab^{-1}b^{-1} = a^3b^{-2}$.

For any $S \subset G$, a word in S will represent an element of G , to see this let $\{1, 2\} = S \subset G = \mathbb{Z}$, then words 1^3 and $2^{-2}1^52$ in S equal $3 \in G$. If a word w in S represents an element $g \in G$ then we write $w =_G g$, so $1^3 =_{\mathbb{Z}} 3$ and $2^{-2}1^52 =_{\mathbb{Z}} 3$.

Definition 2.2. Let $w_1 = s_1s_2\dots s_n$, $w_2 = t_1t_2\dots t_m$ be words in S , such that $s_i, t_i \in S \cup S^{-1}$.

Product

$$(w_1)(w_2) = s_1s_2\dots s_nt_1t_2\dots t_m$$

Remembering to use reduction on $s_1s_2\dots s_nt_1t_2\dots t_m$ when necessary.

Inverse

$$(w_1)^{-1} = s_n^{-1}s_{n-1}^{-1}\dots s_1^{-1}$$

It isn't uncommon to see the inverse of generators represented differently, typically by an overline or converting the case, $a^{-1} = \bar{a}$ and $a^{-1} = A$ respectively.

We now have the prerequisites to define a generating set.

Definition 2.3. *Generating Set.* A generating set of a group is a subset of the group, such that every element of the group can be represented by a word in the subset. More formally:

Let G be a group and $S \subset G$ a subset of elements of G . S is a generating set

of G if for any $g \in G$ there is a word w in S such that $w =_G g$. Equivalently S is a generating set of G if $\langle S \rangle = G$.

In the case S is a generating set for G , then we say the elements in S are *generators* of G .

Example

If $G = \mathbb{Z}$, then $\{1\}, \{-1\}, \{2, -3\}$ each generate G but $\{2\}$ does not.

If $G = \mathbb{Z}/10\mathbb{Z}$, $\{7\}, \{9, 1\}$ each generate G but $\{2\}$ does not.

If a group G has a finite generating set S , then we say that G is *finitely generated*. For convenience, it is common practise to assume that S is a symmetric set, that is $S = S^{-1}$, and does not contain the identity. This will be adopted now on unless S is specified otherwise. Given any generating set W a symmetric set S is produced by simply letting $S = W \cup W^{-1}$, in the case W is finite, S will also be finite.

Definition 2.4. *Free group.* For a symmetric set S of symbols, denote the set of reduced words in S as $F(S)$. Then,

$$F(S) = \bigcup_{n=0}^{\infty} \{s_1 s_2 \dots s_n \mid s_i \in S, \forall i \text{ and } s_{i+1} \neq s_i^{-1}, \forall 1 \leq i \leq n-1\}.$$

The reduced word of the product of two elements in $F(S)$ is called the reduced concatenation of those words. $F(S)$ with the binary operation of reduced concatenation results in the free group of S .

The proof that this forms a group is straight forward so is omitted.

Although earlier examples of the Free Group are seen, Dyck is credited with the first paper introducing and developing a lot of the concepts described above and attributing free groups to generating sets and presentations [8]. In fact there are many ways of constructing free groups, a different one is presented by Robinson in [29, Chapter 2]. Free groups themselves have properties not explored too deeply here, a main result being the Nielsen–Schreier theorem, which states that every subgroup of a free group, is free[35]. Generating sets also have properties not explored too deeply, for further reading I point you to chapters 2 and 6 [29].

2.2 Cayley Graphs

For any element in a finitely generated group, there is a reduced word in the generating set that represents the element. In the case of free groups, this word is unique [14], but it is not always the case. For example take the cyclic group of order 3 generated by set $S = \{a\}$, then $a^2 = a^5$ and $a^0 = a^{-5=3} = a^9$ in context of the group. In cases like this it is important to have some notion to decide which word will represent an element within the class of words equal to it.

Definition 2.5. *Word Metric* Let $S \subset G$ a finite symmetric generating set for G . Let $g, h \in G$ and define a metric on G induced by S as such,

$$d_s(g, h) = \min\{n \mid g^{-1}h = s_1s_2\dots s_n, s_i \in S\}$$

That is the minimal number of elements in S such that $g^{-1}h$ is a product of those elements.

Example

For $G = \mathbb{Z}$ and $S = \{1, -1\}$ then $d_s(g, h) = |h - g|$ and $|g|_s = |g|$. However if you took another generating set say $S' = \{2, 3\}$ then the formula would not be simple.

For $G = C_3$ and $S = \{a, a^{-1}\}$ then $d_s(g, h) = 1$ if $g \neq h$, otherwise 0.

Proposition 2.6. d_s is a metric on G .

Proof. Let $S \subset G$ a finite symmetric generating set for G . Let $g, h \in G$ and let d_s be as in 2.5. First it is given that S is a generating set, meaning there exists $g^{-1}h = s_1s_2\dots s_n, s_i \in S$ with $n \in \mathbb{N}$. This shows that d_s maps to $\mathbb{R}$ with domain $G \times G$.

Let $d_s(g, h) = 0$, then $g^{-1}h$ has length 0, thus equals the identity element and $h = g$.

Let $d_s(g, h) = n$ then $g^{-1}h = s_1s_2\dots s_n$. Taking inverses we see that $(g^{-1}h)^{-1} = (s_1s_2\dots s_n)^{-1}$ giving us $h^{-1}g = s_n^{-1}\dots s_2^{-1}s_1^{-1}$, thus $d_s(h, g) \leq n = d_s(g, h)$. By repeating but with switched h and g we achieve $d_s(g, h) \leq d_s(h, g)$, giving us equality.

Let $g, h, k \in G$ and $d_s(g, h) = n, d_s(h, k) = m$. Then $g^{-1}h = s_1s_2\dots s_n$ and $h^{-1}k = t_1t_2\dots t_m$ with $s_i, t_i \in S$. Thus $s_1s_2\dots s_nt_1t_2\dots t_m = g^{-1}hh^{-1}k = g^{-1}k$, showing that $d_s(g, k) \leq n + m$. $\square$

This leaves us with $d_s(e, g)$ being the shortest way of representing g as a word in S . Let us denote this as $|g|_s$, the *word length* of g with respect to S . If for any element in the group, we choose the word $s_1s_2\dots s_n$ that produces $|g|_s$ to represent it, then we have a *normal form* of that group, with the word chosen being the shortest and simplest. Apply this to the cyclic group of order 3 and we end up with the words $\{e(a^0), a^1, a^{-1}\}$ representing the elements. We also have produced a metric between elements in a group, for any group that is finitely generated.

Applying the word metric isn't without issue however, the *word problem* was introduced by Dehn early into the study of finitely generated groups [35]. Essentially the issue is with deciding if two words from a set of generators represent the same element in the group, although presented slightly differently by Dehn. He states that an algorithm with finite steps should be given in order to solve this problem, something he achieved for surface groups with *Dehn's Algorithm* [35]. It was shown by Novikov, and later Boone, however that there exists a finitely generated group that has no such

algorithm [30]. Fortunately Dehn's Algorithm and the Knuth-Bendix completion algorithm help solve the word problem, especially in the context on small cancellation groups. So although not always possible, it is feasible to find the equivalent element, and hence its word length, for a class of words.

Measuring the growth rate of a group has its own history and took off in popularity along with the study of hyperbolic groups in the 1900's [16]. In short it measures the cardinality of a ball of increasing radius about the identity and can allow us to specify between groups or measure similarities within generating sets of groups.

Definition 2.7. *Growth Function* Let $S \subset G$ a finite symmetric generating set for G . We define the function

$$\gamma_{G,S}(N) := |\{g \in G : |g|_S \leq n\}|$$

and call this the *growth function of G with respect to S* .

Example

Let $S = \{a\}$. $\gamma_{F(S),S}(n) = 1 + 2n$, as the ball expands in the a and a^{-1} directions with each radius increase.

With such a vast history and importance in studying groups, the growth function of groups has a multitude of useful and interesting properties. In particular a Free group $F(S)$ with $|S| = n > 1$ has

$$\gamma_{F(S),S}(N) = \frac{n(2n-2)^{k-1} - 1}{n-1},$$

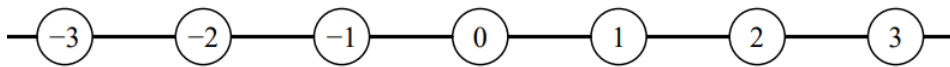
an upper bound of $\gamma_{G,S}(N)$ for groups with the same generating set [32]. The growth rate was particularly important and catered for in subsection *Word Generation*.

Definition 2.8. *Cayley Graph* Let $S \subset G$ a finite symmetric generating set for G . We let $\Gamma(G, S)$ denote the *Cayley graph of G with respect to S* , sometimes also denoted $\text{Cay}(G, S)$. $\Gamma(G, S)$ is the metric graph with vertex set G and whose edge set is $\{\{g, gs\} | g \in G, s \in S\}$.

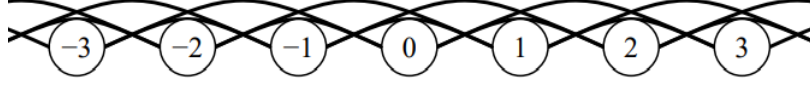
As S is a generating set for G , every element of G can be represented as a word in S , therefore $\Gamma(G, S)$ is a connected graph.

Example

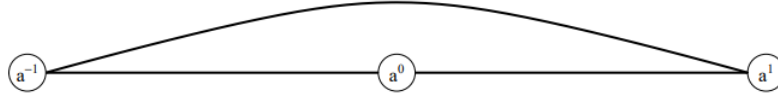
Let $G = \mathbb{Z}$ and $S = \{1, -1\}$.



Let $G = \mathbb{Z}$ and $S = \{2, 3\}$.



Let $G = C_3$ and $S = \{a\}$.



Lemma 2.9. *Let $S \subset G$ a finite symmetric generating set for G . $\Gamma(G, S)$ is a proper geodesic metric space.*

Proof. Recalling that as $\Gamma(G, S)$ is a metric graph, each edge is a copy of $[0, 1]$ thus has length 1 and the metric is the inherited from this. In the case of Cayley graphs, the distance between two vertices is simply the smallest number of edges between them. With the vertices being elements of G as words, this is equal to the word metric and does exist as $\Gamma(G, S)$ is connected.

If a point lies on the interior of an edge, not equal to a vertex, then we have two vertices we can reach by travelling along the edge, travelling to either having a distance < 1 . From here we can then travel to any other vertex, with distance as above. The distance of two points on the interior of an edge is then equal to minimum distance of travelling to a neighbouring vertex of the first point plus the distance from there to a neighbouring vertex of the second point plus the distance to the second point from its neighbouring vertex. Each point is connected to 2 vertices, resulting in there being 4 total possible paths that the minimum distance is to be taken from.

This gives way to $\Gamma(G, S)$ being a geodesic metric space, with the geodesic between points being the path that generates the distance as defined above.

$\Gamma(G, S)$ has a finite generating set, every vertex has finite edges. Any closed ball is then a union of finite subsets of edges. Each of these edges being a copy of $[0, 1]$, hence are compact. Any closed ball is then a union of finitely many compact subsets and so is compact. $\Gamma(G, S)$ is also a proper metric space. $\square$

The Cayley graph also allows another way of reaching 2.7 where $\gamma_{G,S}(N)$ is equal to the number of vertices reachable by n edges from the identity.

2.3 δ -Slim Triangles and Hyperbolicity

This section will discuss the hyperbolicity of metric spaces and expand this to a definition for the hyperbolicity of finitely generated groups through *δ -slim triangles*. It will mostly follow [22] and the work of M. Gromov [17], summarised in chapter 1 of [9]. We will end this section with a definition for the hyperbolicity of a finitely generated group in terms of words and its Cayley Graph. There are many other equivalent ways of constructing a definition for a hyperbolic group but this route was chosen due its context in small cancellation groups. A lot of these construction are covered in Gromov's piece [17]. The study of hyperbolic groups is vast and very active, with the maths its built it having its own deep foundation, this sub-section is far from a comprehensive study into the area but instead gives relevant enough information to proceed.

Definition 2.10. *Gromov Product.* Let (X, d) be a metric space and $x, y, o \in X$. The *Gromov product* of x and y at o , denoted $(x, y)_o$, is

$$(x, y)_o = \frac{1}{2}(d(o, x) + d(o, y) - d(x, y)).$$

Proposition 2.11. *The Gromov product satisfies the following:*

- $(x, y)_o = (y, x)_o$
- $d(x, y) = (x, o)_y + (y, o)_x$
- $(x, y)_0 \leq \min\{d(x, 0), d(y, 0)\}$

The proof follows simple computation and the definition.

The Gromov product is essentially a way of measuring angles between x and y with respect to a base point o by measuring how long the geodesics $[0, x]$ and $[0, y]$ follow the same path, although the definition does not depend on the space X being geodesic. A larger Gromov product would suggest that x and y are close with respect to 0 , while a smaller number suggesting more distance between x and y . If $(x, y)_o = 0$ then o is a point on the path $[x, y]$ with $[o, x]$ and $[o, y]$ diverging immediately from o . Our first measure of hyperbolicity follows from how the Gromov product behaves for arbitrary points.

Definition 2.12. *δ -hyperbolic metric space.* Let X be a metric space. If $\exists \delta \in \mathbb{R}_{\geq 0}$ such that the *four-point condition*

$$(x, y)_0 \geq \min\{(x, z)_o, (y, z)_0\} - \delta$$

is satisfied for all $x, y, x, o \in X$ then we say X is *δ -hyperbolic*.

In this case we also say that X is $H_1(\delta)$, introducing such notation now to separate different hyperbolic definitions between similar spaces later.

Example

Any bounded metric space X is $H_1(\delta)$ for $\delta = \max\{d(x, y) | x, y \in X\}$.

$\mathbb{R}$ with the Euclidean metric is $H_1(0)$.

Any metric space X with a new metric $d(x, y)' = \log(1 + d(x, y))$ [17].

Any $\mathbb{R}^n$ with $n > 1$ is *not* δ -hyperbolic.

The choice of base point may seem important due to the relation to the Gromov product but the definition is actually independent of the base point. This is due to result from the following lemma.

Lemma 2.13. *Let X be a metric space. Suppose there exists some δ and fixed base point $o' \in X$ such that*

$$(x, y)_{o'} \geq \min\{(x, z)_{o'}, (y, z)_{o'}\} - \delta$$

for all $x, y, z \in X$. Then for any $t \in X$

$$(x, y)_{o'} + (z, t)_{o'} - \min\{(x, z)_{o'} + (y, t)_{o'}, (x, t)_{o'} + (y, z)_{o'}\} \geq -2\delta$$

Proof. This is shown by computation on the inequalities in 2.13 resulting in

$$(x, y)_{o'} + (z, t)_{o'} - \min\{(x, z)_{o'} + (y, t)_{o'}, (x, t)_{o'} + (y, z)_{o'}\} =$$

$$(x, y)_t - \min((x, z)_t, (y, z)_t) \geq -2\delta$$

□

A much more verbose and intuitive proof can be found in [15].

Corollary 2.14. *Let X be a metric space. Suppose there exists some δ and fixed base point $o' \in X$ such that*

$$(x, y)_{o'} \geq \min\{(x, z)_{o'}, (y, z)_{o'}\} - \delta$$

for all $x, y, z \in X$. Then X is $H_1(2\delta)$

Definition 2.15. *Geodesic triangle.* Let X be a geodesic metric space and $x, y, z \in X$. A concatenation of the 3 geodesics, $[x, y]$, $[x, z]$ and $[y, z]$, between the points is called a *geodesic triangle* in X .

By abusing notation we will denote a geodesic triangle between points x, y and z by $[x, y, z]$.

Depending on the nature of the individual geodesics, the sides of the triangle, $[x, y]$, $[x, z]$ and $[y, z]$, may intersect in places other than x, y and z .

δ -slim triangle

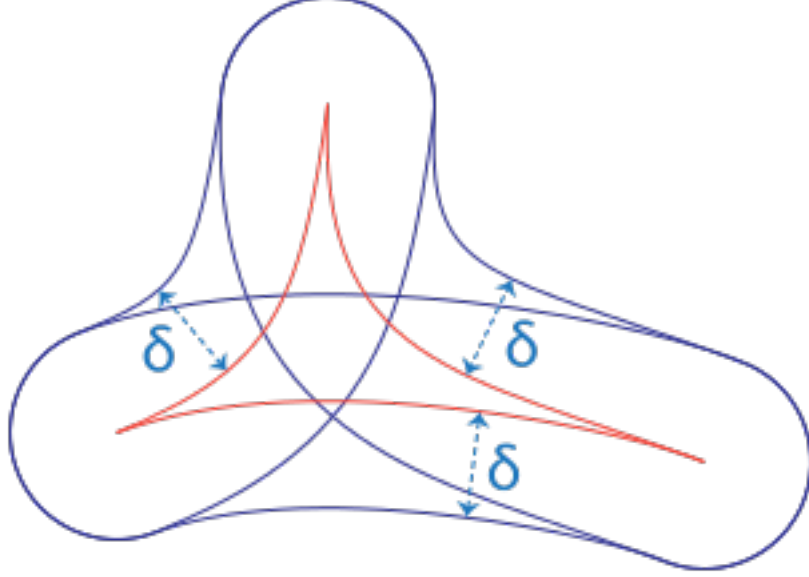


Figure 1: A δ -slim triangle, source = https://upload.wikimedia.org/wikipedia/commons/4/46/Delta_thin_triangle_condition.svg

Definition 2.16. *Slim triangles.* Let X be a geodesic metric space and $[x, y, z]$ a geodesic triangle in X . $[x, y, z]$ is called δ -slim if each side is contained within a δ neighbourhood of the remaining sides. More formally, let $a \in [x, y]$, then there exists $b \in [y, z] \cup [x, z]$ such that $d(a, b) \leq \delta$. Note that the choice of $[x, y]$ is arbitrary and the inequality must hold for any point on any side of the triangle.

The definition of triangles within a geodesic metric space can give us a more geometric approach to defining hyperbolicity and is mostly necessary to continue. It isn't uncommon to see triangles introduced first and used to define all notions of hyperbolic spaces. The Gromov product itself can be defined via internal points in a triangle, this geometric approach (and a few others) is seen in chapter III.H of [7].

Definition 2.17. *δ -hyperbolic geodesic metric space.* Let X be a geodesic metric space. X is called δ -hyperbolic, or Rips-hyperbolic, if all geodesic triangles of X are δ -slim for some $\delta \in \mathbb{R}_{\geq 0}$.

We will denote this as $H_{Rips}(\delta)$, to specify between the hyperbolicity defined in 2.12. We refer to this as Rips-hyperbolic due to Gromov attributing the definition to the previous work of Rips.

Proposition 2.18. *Let X be a geodesic metric space. X is Rips-Hyperbolic if and only if it is $H_1(\delta)$ for some $\delta \in \mathbb{R}_{\geq 0}$.*

Proof. The proof is omitted as it requires introduction to some properties irrelevant to the overall topic and does not include much information important to the work. A proof with similar to context seen can be found in [15], with chapter III.H of [7] giving a different method of proof. $\square$

You can see from figure 1 that the smaller the δ in a slim triangle, the closer the triangle is to a tripod. When $\delta = 0$, the triangle is exactly a tripod, thus a $H_{Rips}(0)$ space is exactly a tree. The *delta* can be considered a measure of how tree-like a geodesic metric space is. More accurately, given the roots in hyperbolic geometry, these hyperbolicity definitions help classify spaces that have properties similar to the hyperbolic plane. Following this thinking and applying 2.18 to the examples for 2.12, we get a better geometric understanding of hyperbolicity of spaces. We can also see that the Cayley Graph $\Gamma(F(S), S)$ for any finite S is 0-hyperbolic in the sense of Rips, leading us to Gromov's definition for hyperbolicity of Groups.

Definition 2.19. *Hyperbolic group.* Let G be a group. G is a *hyperbolic group* if G has a generating $S \subset G$ and $\Gamma(G, S)$ is a Rips-hyperbolic space.

In this case we may also refer to G as *word-hyperbolic*, *Gromov-hyperbolic* or *hyperbolic in the sense of Gromov*.

Example

It is easy to see from the Cayley Graph of $\mathbb{Z}$ that it is hyperbolic.

The free group $F(S)$ for any S is hyperbolic.

The fundamental group of a genus-2 surface is hyperbolic, a famous example. $\langle a, b, c, d | a^2 b^2 c^2 d a^{-2} b^{-2} c^{-2}, b^2 d^2 a^2 c b^{-2} d^{-2} a^{-2} \rangle$ presents a hyperbolic group. $\mathbb{Z}^n$ is not hyperbolic for any $n > 1$.

There are many more definitions and measures of hyperbolicity for metric spaces with a myriad of interesting theorems and studies behind them. As this article only concerns itself with the connection with hyperbolic boundaries and small cancellation theory, we have gave a brief and far from detailed look into the area. Part 1, Chapter 1 of [19], gives a more thorough summation of hyperbolic metric spaces, and the important equivalence between certain concepts of δ -hyperbolicity.

From definition 2.19 it appears that G being hyperbolic depends on the choice of S for its Cayley graph. This is not the case however, but to explore why we must first introduce the idea of a quasi-isometry.

We will briefly recall the definitions of isometries.

Definition 2.20. Let (X, d_X) and (Y, d_Y) be metric spaces and $f : X \rightarrow Y$ a map such that $d_Y(f(a), f(b)) = d_X(a, b)$ for all $a, b \in X$. Then f is an isometric embedding.

If f is surjective then it is an *isometry* and (X, d_X) and (Y, d_Y) are *isometric*.

This definition is quite rigid and it is, at times, important to want to generalise the idea of an isometry, giving us a broader look at the properties of similar spaces. An example we've already seen is when comparing Cayley graphs of a group from two distinct generating sets. The need for this gives rise to quasi-isometries.

Definition 2.21. *Quasi-isometric embedding.* Let (X, d_X) and (Y, d_Y) be metric spaces and $f : X \rightarrow Y$, a map, such that

$$\frac{1}{K}d_X(a, b) - C \leq d_Y(f(a), f(b)) \leq Kd_X(a, b) + C$$

for $K \geq 1$, $C \geq 0$ and $a, b \in X$. Then f is a (K, C) -*quasi-isometric embedding*.

In cases where $C = 0$, a $(K, 0)$ -quasi-isometric embedding, the resulting map is called a *K-bi-Lipschitz embedding*. Such as before when weakening the condition of the embedding, we also want to weaken the dependency on a surjection for two spaces to be isometric.

Definition 2.22. *Coarse surjective.* Let (X, d_X) and (Y, d_Y) be metric spaces and $f : X \rightarrow Y$ a map. If for all $y \in Y$, there exists $x \in X$ such that

$$d_Y(y, f(x)) \leq D$$

with $D \geq 0$, then f is D -coarsely-surjective. Every element y in Y is at most D away from an element $f(x)$ in Y .

With the relaxation of the parameters of an isometry, we can now introduce the notion of two spaces being quasi-isometric to each other and thus have some form of comparison for roughly similar spaces.

Definition 2.23. *Quasi-isometry.* Let (X, d_X) and (Y, d_Y) be metric spaces and $f : X \rightarrow Y$, a map, such that f is a (K, C) -quasi-isometric embedding and D -coarsely-surjective. Then f is a *quasi-isometry* and X is *quasi-isometric* to Y .

If there exists a quasi-isometry from X to Y then a quasi-isometry also exists from Y to X [6]. We will therefore refer to two spaces as being quasi-isometric if one such mapping exists.

Example

$\mathbb{R}$ is $(1, 1)$ -quasi-isometric to $\mathbb{Z}$.

The identity map from $\mathbb{R}^2$ with the Euclidean metric to $\mathbb{R}^2$ with the Manhattan distance is a $(\sqrt{2}, 0)$ embedding.

There exists no quasi-isometric embedding for $\mathbb{Z}^m$ and $\mathbb{Z}^n$ if $m \neq n$.

With this definition we now have a capability of comparing spaces not covered by typical isometries. We have also attached three constants, K, C and D to these as some form of measure of similarity. As K, C and D increase, the spaces become more dissimilar, especially on the small scale. When comparing Cayley graphs of groups generated from different sets we come across an important result.

Lemma 2.24. *Let $S, S' \subset G$ be finite generating sets for a group G . Then $\Gamma(G, S)$ is quasi-isometric to $\Gamma(G, S')$.*

Proof. Let $f : \Gamma(G, S) \rightarrow \Gamma(G, S')$ map vertices to themselves and edges to either vertex it connects. As edges in $\Gamma(G, S')$ have length 1, the mapping from edges to a vertex have at most distance 1. To prove that vertices are quasi-isometry recall that both S and S' are generating sets. Therefore there exists some K such that every element of S can be written as less than K elements of S' . Equally every element of S' can be written as less than K elements of S . The relative distance between the same vertex on the two graphs is therefore up to this multiplicative K . Resulting in the map f being a $(K, 1)$ -quasi-isometry, with $D = 1$ coarse-surjectivity. $\square$

Theorem 2.25. *Let X and Y be quasi-isometric geodesic metric spaces. X is hyperbolic iff Y is hyperbolic.*

Proof. The proof is omitted, reasons similar to the proof of 2.18. A proof can be found in [15]. $\square$

With this we can now come lead back to our issue with generating sets. At first glance it would appear that the hyperbolicity of a group depends on its choice of generating set. This however is not the case. By the combination of 2.24 and 2.25 we see that if $\Gamma(G, S)$ is Rips-hyperbolic for some finite generating set S , the Cayley graph for any generating set is also Rips-hyperbolic, albeit δ will change. Therefore 2.19 does not depend on the choice of S .

2.4 Gromov Boundaries

To evaluate a hyperbolic groups points at infinity is to evaluate its boundary. Studying the boundary is fundamental to the overall study of hyperbolic groups and can be extremely useful, with numerous theory and application [25]. A natural topology arises from the boundary of a hyperbolic group, it is invariant under quasi-isometry, and the action of a hyperbolic group on its boundary is uniform convergence action [25], [4]. These rich properties are why we concern ourselves with the boundary and wish to produce a method of estimating and visualising it. This section will discuss the definitions and properties of boundaries for hyperbolic groups, mostly following work in chapter III.H of [7] and chapter 1 of [19].

Although hyperbolicity is defined for metric spaces, as we concern ourselves with hyperbolic groups for which the Cayley graphs are proper geodesic metric spaces, we will assume from now that all spaces are geodesic as well and construct boundaries from this perspective.

Suppose for a metric space X we have two geodesic rays $\gamma_1 : [0, \infty) \rightarrow X$ and $\gamma_2 : [0, \infty) \rightarrow X$. If there exists some C such that for all $t \geq 0$ we have $d(\gamma_1(t), \gamma_2(t)) < C$, then we say that γ_1 is equivalent to γ_2 , writing $\gamma_1 \sim \gamma_2$. We will denote the equivalence class of a geodesic ray γ as $[\gamma]$.

Given a starting point, a boundary of a space is then the set of equivalence classes of geodesic rays emanating from this point.

Definition 2.26. *Geodesic boundary.* Let X be a δ -hyperbolic geodesic metric space. Then

$$\partial X = \{[\gamma] \mid \gamma \text{ is a geodesic ray in } X\}$$

is the boundary of X .

In literature it is common to see another definition for boundary of X , done via sequences of points. Given a sequence that tends to a point p , for p to exist on the boundary it must be that the distance from the sequence and a base point tends to infinity as the sequence does so.

Definition 2.27. Let X be a δ -hyperbolic geodesic metric space with base point $o \in X$. If for a sequence (x_i) with $x_i \in X$, we have

$$\lim_{i,j \rightarrow \infty} (x_i, x_j)_o = \infty$$

then (x_i) converges to infinity.

In this case (x_i) is sometimes called a *Gromov sequence*. Although this definition requires a base point o , the choice of o is not important [25].

We now define a relation on Gromov sequences, (x_i) and (y_i) , as such

$$(x_i) \sim (y_i) \iff \lim_{i,j \rightarrow \infty} (x_i, y_j)_o = \infty.$$

Given that the Gromov product measures in a way the length from o to the geodesic $[x, y]$, if $(x_i, y_j)_o$ tends to infinity as the sequences do it follows that both sequences are on the same geodesic ray, or on difference rays within the same equivalence class. If they were not on the same ray then the geodesic $[x_i, y_i]$ would get larger, but its closest point to o would remain the same.

In general this relation is symmetric and reflexive and much like sequences that converge to infinity, does not depend on the choice of base point. Fortunately in the case of hyperbolic spaces, it is also transitive, a simple proof displayed in chapter 1 of [19] and follows from properties of $H_1(\delta)$. As we only concern ourselves with these spaces then for our purposes the relation is an equivalence relation, allowing us to denote $[(x_i)]$ as the equivalence class.

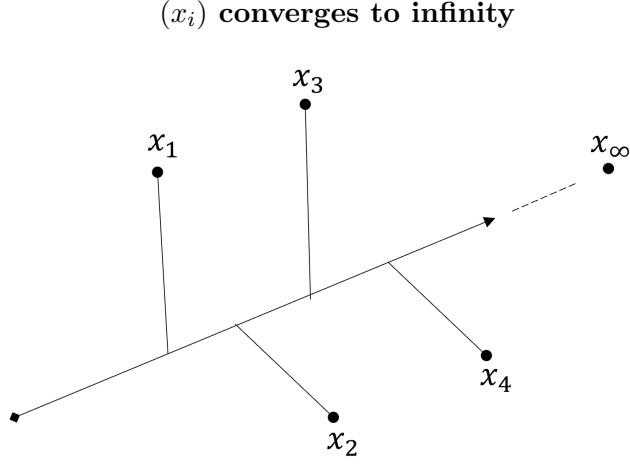


Figure 2: An illustration of a sequence (x_i) that converges that infinity for a hyperbolic metric group.

Definition 2.28. *Visual boundary.* Let X be a δ -hyperbolic geodesic metric space. Then

$$\partial_s X = \{[(x_i)] \mid (x_i) \text{ is a sequence that converges to infinity}\}$$

is the boundary of X .

It may not be that $\partial_s X = \partial X$ but if the spaces in question are proper and geodesic, then a bijection exists between the two. This can be seen as if a sequence (x_i) converges to infinity then the geodesic $[o, x_\infty]$ is a geodesic ray. Similarly if we have a geodesic ray γ , then the sequence $(\gamma(t))$, $t \in \mathbb{R}^+$ converges to infinity. We can now expand the definition from hyperbolic spaces to hyperbolic groups.

The boundary of a hyperbolic group is the boundary of the Cayley graph associated to the group. Recalling that the boundary is quasi-isometry invariant so the choice of Cayley graph is unimportant.

Example

If $G = \mathbb{Z}$ then the Gromov boundary of G is the set $\{\infty, -\infty\}$.

Let $\|S\| \geq 2$ and $G = F(S)$, then ∂G is the Cantor set [25].

Let G be finite, then $\partial G = \emptyset$.

As we aim to estimate the boundary of a hyperbolic group, we will assume from now that the groups in question are infinite.

These are not the only constructions for the boundary of a hyperbolic metric space. Common constructions include those from quasi-geodesics, or

from geodesic rays with a fixed base point, both seeing similar definitions to 2.26. For hyperbolic metric spaces however the maps between them are bijective, with each of the models having its own use. For example, construct the boundary as in 2.26 but with each geodesic ray γ having $\gamma(0) = x$ for some $x \in X$. By connecting any two points p, p' in ∂X via the geodesic rays between them and x we obtain the following lemma [22].

Lemma 2.29. *For any $p, p' \in \partial X$, there exists a bi-infinite geodesic $\gamma : (-\infty, \infty) \rightarrow X$ with $\gamma(-\infty) = p$ and $\gamma(\infty) = p'$.*

The metrics we've discussed so far break down when applied points in the boundary of a group. The distance metric of any two points in the boundary must be infinity, by construction. Regardless of base point, the Gromov product follows suit in breaking down. To construct and evaluate metrics on the boundary of a hyperbolic space we need to extend Gromov's product to account to account for it. It is simpler to do so considering $\partial_s X$.

Definition 2.30. *The Gromov product at infinity.* Let X be a δ -hyperbolic geodesic metric space with basepoint $o \in X$ and $\bar{X} = X \cup \partial_s X$. For sequences (x_i) and (y_i) such that $\xi = [(x_i)]$ and $\eta = [(y_i)]$ with $\xi, \eta \in \partial_s X$ then

$$(\xi, \eta)_0 := \inf \left\{ \lim_{i \rightarrow \infty} \inf(x_i, y_i)_o \right\}$$

taken over all pairs of the sequences (x_i) and (y_i) .

For clarity let $\xi, \eta \in \partial_s X$ as above and $z, w \in X$, then

- $(\xi, \eta)_0$ is defined prior.
- $(\xi, z)_0 = \inf \{ \lim_{i \rightarrow \infty} \inf(x_i, z)_o \}$
- $(w, z)_0$ follows the formula from 2.10.

Corollary 2.31. $(x, y)_0 = \infty \iff x = y \in \partial_s X$.

Proof. This proof follows [19, Chapter 1]. The left to right implication is as follows. Let $(x, y)_0 = \infty$, with $x, y \in X \cup \partial_s X$. Certainly if $x, y \in X$ then $(x, y)_0$ is finite. Suppose then that, without loss of generality, $x \in X$ and $y \in \partial_s X$. Then $(y, x)_0 = \inf \{ \lim_{i \rightarrow \infty} \inf(y_i, x)_o \}$ over the sequence $(y_i) \rightarrow_i y$. Giving us $(y_i, x)_o \leq \min\{d(y_i, 0), d(x, 0)\} \leq d(x, 0)$ (from 2.11) which is finite. Therefore neither point can be an element in X . Thus we suppose $x, y \in \partial_s X$ with $a_i, x_i \rightarrow x$ and $b_i, y_i \rightarrow x$. Giving us $\lim_{i \rightarrow \infty} \inf(a_i, b_i)_0 \geq \inf \{ \lim_{i \rightarrow \infty} \inf(x_i, y_i)_o \} = \infty$. It must be that $\inf(a_i, b_i)_0 = \inf$ and $x = y$

The right to left implication follows directly from the definition 2.30. $\square$

Note that in some literature the definition is

$$(\xi, \eta)_0 := \sup \lim_{i,j \rightarrow \infty} \inf(x_i, y_j)_o.$$

Chapter 3.4 of [10] discusses the difference, showing that the choice of *sup* or *inf* is not important.

This may seem a complicated definition, ideally the simple limit $(x, y)_0 := \lim_{i \rightarrow \infty} (x_i, y_i)_o$ would suffice. Chapter III.H of [7] and chapter 1 of [19] both showcase the same example displaying the issues with the simpler limit and the need for such a definition.

We will apply this extension to define a visual metric on the boundary, constructed in a way to emulate the Euclidian metric on a ball [10, Chapter 3]. As our end goal is to produce a plot in a Euclidian space, already having some method to roughly imitate the Euclidian metric is a considerable step towards estimating the boundary of a hyperbolic group.

To begin we take $\xi, \eta \in \partial_s X$ and a base point $o \in X$ then define

$$p_{o,\epsilon}(\xi, \eta) = e^{-\epsilon(\xi, \eta)_o}$$

for some $\epsilon > 0$. We can consider $p_{o,\epsilon}$ as a measure of separation for points in the boundary. Adopting convention that $e^{-\infty} = 0$, therefore $p_{o,\epsilon}(\xi, \eta) = 0 \iff \xi = \eta \in \partial_s X$ we can see properties of a metric space emerging. Even though $p_{o,\epsilon}$ is also symmetric a bit more work is necessary as it is not always a given that $p_{o,\epsilon}$ satisfies the triangle inequality. We define a pseudo-metric

$$d_{\epsilon,0}(\xi, \eta) = \inf \left\{ \sum_{i=1}^n p_{\epsilon,o}(\xi_{i-1}, \xi_i) \right\}$$

where the infimum is taken over all finite sequences (ξ_i) such that $\xi_o = \xi$ and $\xi_n = \eta$, [7, Chapter III.H].

Definition 2.32. *Visual metric.* Let X be a δ -hyperbolic geodesic metric space with basepoint $o \in X$ and metric d on ∂X . If for all $\xi, \eta \in \partial X$ there exists $k_1, k_2 \in \mathbb{R}^+$ and *visual parameter* a such that

$$k_1 a^{-(\xi, \eta)_0} \leq d(\xi, \eta) \leq k_2 a^{-(\xi, \eta)_0}$$

then d is a visual metric.

Proposition 2.33. *Let X be a $H_1(\delta)$ metric space with basepoint $o \in X$. For $\epsilon \leq \frac{\ln(\sqrt{2})}{\delta}$, $d_{\epsilon,0}$ is a visual metric.*

Proof. The inequality $(x, y)_0 \geq \min\{(x, z)_o, (y, z)_o\} - \delta$ from 2.12 still holds, [19, Lemma 4.6]. Raising to $e^{-\epsilon}$

$$\begin{aligned} e^{-\epsilon(x, y)_0} &\leq e^{-\epsilon(\min\{(x, z)_o, (y, z)_o\} - \delta)} \\ &\leq 2 \max\{p_{o,\epsilon}(x, z), p_{o,\epsilon}(z, y)\}. \end{aligned}$$

Applying [31, Theorem 1.2] as shown in [10, Chapter 3] yields the desired result.

□

Chapter III.H of [7] presents a different method of proof, albeit with slightly different constraints on ϵ , as does [22].

Even though we can define a hyperbolic group, discuss its boundary and visual metric, is it feasible to determine if an arbitrary group is hyperbolic? Can we produce a Rips-hyperbolic space given a generating set? The goal is to estimate the boundary of a hyperbolic group but it is difficult to do so if we do not have an easy method for obtaining hyperbolic groups. The next section aims to explore this and give a succinct method of producing groups we can guarantee are hyperbolic.

3 Small Cancellation Theory

The structure of this article would suggest that small cancellation is a result of Gromov's work on hyperbolic groups. This is not the case however with small cancellation and work around it acting as a basis for Gromov's work [25]. This section will discuss the main ideas regarding small cancellation theory, the links to hyperbolic groups and how Dehn's algorithm played a pivotal part in the development of both.

The study of combinatorial group theory is the basis for a lot of results in this section, with Dehn's work in 1912 acting as a basis for much of the subjects development [7]. In working with closed orientable surfaces and low-dimensional manifolds, Dehn was able to make the connection to the fundamental theory of groups, specifically for infinite groups with finite generators. In doing so he released three problems, the word, conjugacy and isomorphism problems (stated below). This gave way to a lot of development in the mid 1900's [8] with the aim of solving these problems and exploring their characteristics on finitely presented groups. A lot of the material in this section being a direct result of the attempts and advances on Dehn's work and problems. Tartakovskii formalised a lot of the results for small cancellation theory in 1949, followed by Greendlinger developing small cancellation theory as we know today [8, Chapter II.9]. An important result of his stating that finitely presented groups satisfying the $C'(1/6)$ small cancellation condition have word problem solvable by Dehn's algorithm, a similar conclusion Gromov made regarding hyperbolic groups.

Stillwell's translation of Dehn's three algorithmic problems as they were first presented [34].

1. *The word (identity) problem.* An element of the group is given as a product of generators. One is required to give a method whereby it

may be decided in a finite number of steps whether this element is the identity or not.

2. *The conjugacy (transformation) problem.* Any two elements, s and t of a group are given. A method is sought for deciding the question whether S and T can be transformed into each other, i.e. whether there is an element u such that $s = utu^{-1}$.
3. *The isomorphism problem.* Given two groups, one is to decide whether they are isomorphic or not (and further, whether a given correspondence between the generators of one group and elements of the other is an isomorphism or not).

It is clear that the first two problems are very closely related, the word problem can actually be considered a special case of the conjugacy problem, concerning itself only with the identity instead of the conjugacy class of a general element. The ordering also follows the difficulty, with the isomorphism problem being famously incredibly difficult and typically completely unsolvable [34, Chapter 1].

3.1 Group Presentations

The presentation of a group is a method of specifying a group and its properties in a simplistic and straightforward manner. This is done with a set of generators and a list of relations on words in those generators. Every group has a presentation, with a lot having many. Presenting a group in this way leads to the ability to define the groups properties only on some properties of its relations. The bulk of work in this subsection being accredited to Dyck in the 1880's, and acting as a foundation for combinatorial group theory since [8]. This subsection mostly follows from chapter 1 of [24].

Definition 3.1. *Presentation.* Suppose we have

- S a finite set.
- $F(S)$ the free group of S .
- A subset $R \subset F(S)$.
- $\langle\langle R \rangle\rangle$, the normal closure of R in $F(S)$.

Then if we have a group $G \cong F(S)/\langle\langle R \rangle\rangle$, $\langle S | R \rangle$ is a *presentation* of G . S is called the set of *generators* of G and R the set of *relations*. If both S and R are finite, then G is *finitely presented*.

This at first appears difficult and a cumbersome definition. If we consider S a generating set as before, and R a list of rules of the group then it becomes more clear, especially with examples.

Example

$\langle S \rangle$ is the Free Group of S , named such as it is free of any relations, in a way the simplest the group you can produce with S .

$\langle a | a^n = 1 \rangle$, is the cyclic group of order n .

The convention is that terms in R with no equals sign are equal to the group identity, represented with 1, thus $\langle a | a^n = 1 \rangle \rightarrow \langle a | a^n \rangle$.

$\langle a, b | ab = ba \rangle$ is a representation for $\mathbb{Z}$.

$\langle a, b | [a, b], a^5, b^7 \rangle$ is a representation for $\mathbb{Z}/5\mathbb{Z} \times \mathbb{Z}/7\mathbb{Z}$. Noting that $[a, b] = aba^{-1}b^{-1}$ is called the *commutator* of a and b .

$\langle a_1, b_1, a_2, b_2 | [a_1, b_1][a_2, b_2] \rangle$ represents the fundamental group of a genus-2 surface, a main group of Dehn's studies.

In addition to the convention discussed above, it is also useful to have relations always be a word in S equal to the identity, resulting in R being a subset as is stated the definition. This is easily done as in any case where we have $w_1 = w_2 \in R$, we have $w_1 w_2^{-1} = 1$ and can let $w_1 w_2^{-1} \in R$ instead. R is then a set of words equivalent to the empty word, making it easier to find the norm of a word and to compare relations between themselves.

Note that if any word $w = urv =_G g$, for some (possible empty) words u, v and $r \in R$ a relation, by replacing r with the empty word we get $uv =_G g$. An easy operation to produce a smaller word representing an element of the group. From here on, when discussing a word in the context of a group or presentation, we will assume that this operation has already occurred, similar to how we assumed words are already reduced.

Theorem 3.2. *All groups have a presentation and all finite groups have finite presentations.*

Proof. This seems a natural conclusion given the definition and indeed the proof doesn't complicate things. Suppose we have a group G , then the set of elements in G certainly generate G . Equally the set of relations $R = \{g \cdot h = gh | g, h \in G\}$, spans all of G . Then $\langle G | R \rangle$ is a presentation for G , albeit a very verbose one. The proof that all finite groups have finite presentation easily follows from this. $\square$

Although all groups can be presented, the manner in which the proof states is not particularly helpful. Typically we take the most compact presentation, with the smallest S and R to still describe the group. The presentation is now very useful for smaller finite groups, or groups with few relations that are similar to trees.

Definition 3.3. *Cyclically reduced word.* Let $S \subset G$ a generating set for G and w a word in S . If every cyclic permutation of w is reduced, then w is a cyclically reduced word. If all words in a set are cyclically reduced, then we can refer to that set as cyclically reduced.

Example

$w = aba$ has the set of cyclic permutations $\{aba, ba^2, a^2b\}$, is cyclically reduced.

$w = aba^{-1}$ has the set of cyclic permutations $\{aba^{-1}, baa^{-1}, aa^{-1}b\}$, is not cyclically reduced.

Cyclic reduction is important when we want to consider a word in a set of relations.

Proposition 3.4. *Let $\langle S|R \rangle$ be a presentation with $r \in R$, not cyclically reduced. Then r can be represented and derived by a smaller word as a relation.*

Proof. Let $r = s_1 s_2 \dots s_n = 1$ be a cyclically reduced relation. Then by right action of s_n^{-1} and left action of s_n we get $s_n s_1 s_2 \dots s_{n-1} = 1$. Every cyclic permutation of r equals the identity. It follows then that if we have r' , a relation that is not cyclically reduced, then there exists a cyclic permutation of r' that can be reduced while still equal to the identity. Thus r' should be replaced by this reduced word in R . $\square$

Note that if every element $r \in R$ of a set of relations is cyclically reduced, then we can derive that every cyclic permutation of r also equals the identity, and r is conjugate with each of its cyclic permutations, as shown in the above proof. The inverse of every cyclic permutation of r is also equal to identity, easily shown by taking any $r =_G 1$ and by action (either left or right) of r^{-1} .

We saw previously that changing the generating set, changes the Cayley graph for its group even though these graphs exhibit the same hyperbolic behaviours. Something similar occurs with presentations where a group with a determined set of properties can have many potential presentations, choosing one of these may not be trivial. Tietze transformations aim to alleviate this, they are used to transform a presentation into another presentation, typically of the same group.

Definition 3.5. *Tietze transformations.*

1. *T1, adding a generator.* Sometimes seen as $S+$.

$$\langle S|R \rangle \rightarrow \langle S \cup \{b\} | R \cup \{bw^{-1}\} \rangle$$

where $b \notin S$ and w is a word in S . For example let $G = \langle a | a^8 \rangle$, we add generator $b = a^2$ resulting in $G = \langle a, b | a^8, ba^{-2} \rangle$ where $ba^{-2} = 1 \iff b = a^2$.

2. *T2, removing a generator.* Sometimes seen as $S-$.

$$\langle S|R \rangle \rightarrow \langle S/\{a\} | R/\{aw^{-1}\} \rangle$$

where $a \in S$ and w is a word in S/a . To do this the generator being removed must be a word of the other generators and thus redundant otherwise the group will change. Relations are then adjusted to account for this. For example take $\langle a, b | ba^6, ba^{-2} \rangle$, we can remove $b = a^2$ and adjust to remaining relation to give us $G = \langle a | a^8 \rangle$.

3. T_3 , adding a relation. Sometimes seen as $R+$.

$$\langle S | R \rangle \rightarrow \langle S | R / \{r\} \rangle$$

where $r \in \langle\langle R \rangle\rangle / R$ is a new relation derived from other relations. For example $\langle a | a^8 \rangle \rightarrow \langle a | a^8, a^{-16} \rangle$.

4. T_4 , removing a relation. Sometimes seen as $R-$.

$$\langle S | R \rangle \rightarrow \langle S | R \cup \{r\} \rangle$$

where $r \in \langle\langle R / \{r\} \rangle\rangle \cap R$ and r can be derived from other relations.

Theorem 3.6. *Two finite presentations $\langle S | R \rangle$ and $\langle T | U \rangle$ define isomorphic groups if and only if $\langle T | U \rangle$ can be obtained from $\langle S | R \rangle$ with a sequence of Tietze transformations.*

Proof. It is easy enough to see that each of the Tietze transformations preserve the group presented, the right to left implication follows from this. To prove to left to right we will do the following.

Suppose $\langle S | R \rangle$ presents G , $\langle T | U \rangle$ presents G' , and $f : \langle S | R \rangle \rightarrow \langle T | U \rangle$ is isomorphic. For $t \in T$ take w_t as the word that represents $f^{-1}(t)$ in S . Similarly let w_s be the word $f(s)$ in T for $s \in S$. Apply the following transformations:

$$\begin{aligned} \langle S | R \rangle &\xrightarrow{T_1} \langle S \cup T | R \cup \{tw_t^{-1} | t \in T\} \rangle \\ &\xrightarrow{T_3} \langle S \cup T | R \cup \{tw_t^{-1} | t \in T\} \cup U \rangle \\ &\xrightarrow{T_3} \langle S \cup T | R \cup \{tw_t^{-1} | t \in T\} \cup U \cup \{sw_s^{-1} | s \in S\} \rangle \end{aligned}$$

Equally we can apply the same steps to $\langle T | U \rangle$ and reach the same place. From this point T_2 and T_4 can be applied to reverse transformation, taking us from one representation to the other. $\square$

Despite this theorems apparent ability to show if two presentations are isomorphic, there actually does not exist an algorithm to decide this as remarked in chapter 1 of [24]. In fact doing so is famously very difficult and giving such an algorithm is impossible, even for trivial groups [34]. Despite the age of these references, 1980 and 2007 respectively, it does not appear that much progress has been made regarding this problem. The theorem does help solve the word problem in some regard. From the theorem it follows that if a group with a presentation has a solvable word problem, then so does any other presentation of the same group.

3.2 Small Cancellation

Dehn's algorithm was first introduced when he proved that it can be used to solve the word problem for a special class of surface groups [8, chapter 1]. From here the theorems used in this proof were expanded by M. Greendlinger into the small cancellation theory we know today [8, chapter 1]. He proved that Dehn's algorithm could be applied to any group satisfying certain small cancellation conditions to solve the word problem, which in turn gave a set of examples and basis for Gromov to develop the theory of hyperbolic groups. To begin we will discuss small cancellation theory, building our way up to the connections with hyperbolic groups. This subsection follows definitions from [24, chapter VII] and [27, chapter V] .

Definition 3.7. Let G be a group with finite presentation $\langle S|R \rangle$ with all elements of R cyclically reduced and not derivable from other relations. Denote $\overline{R}$ as the set of all cyclic permutations of elements of R and their inverses, called the *symmetrized closure* of R . We may also say that $\overline{R}$ is *symmetrized*. If for unique words $r_1, r_2 \in \overline{R}$, we have $r_1 = uw_1$ and $r_2 = uw_2$, then u is called a piece.

By taking the product $r_1^{-1}r_2$, as defined above, we see that u is cancelled.

There are many altering definitions of pieces, most stating that a piece can occur anywhere in r_1 and r_2 not necessarily as the initial segment. This is an equivalent definition however as $\overline{R}$ contains all cyclic permutations of elements in R , so if a piece exists in the middle of some word, there is another element in $\overline{R}$ where it is at the start. Pieces are fundamental to the study of small cancellation theory, with it being based around groups with presentations that put restrictions on how pieces behave and occur in the set of relations. Depending on the conditions met by pieces, we can derive a lot of important properties on the group presented [27, chapter V]. Equally we can work in reverse and construct a presentation, hence a group, that has a set of properties we desire.

Some properties of a group G that can be inferred from small cancellation theory are:

- If G is infinite.
- Which elements $g \in G$ are torsion elements. That is they have finite order so $g^n = 1$.
- If a non-cyclic free subgroup $H \leq G$ exists.

and many more, as shown in chapter VII of [24].

The most common condition, $C'(\lambda)$, $0 < \lambda < 1$, takes a metric form to assert that pieces are of a relative size to the elements in $\overline{R}$.

Definition 3.8. $C'(\lambda)$ *small cancellation condition*. Let G be a group with finite presentation $\langle S|R \rangle$ with all elements of R cyclically reduced and not derivable from other relations, with R also *symmetrized*. If for any $r = uw_1 \in R$ where u is a *piece*, $\langle S|R \rangle$ satisfies the small cancellation condition $C'(\lambda)$ if

$$|u|_s < \lambda|r|_s.$$

Another, related condition is $C(n)$, a non-metric condition where $n \in \mathbb{N}$.

Definition 3.9. $C(n)$ *small cancellation condition*. Let G be a group with presentation as in 3.8. $\langle S|R \rangle$ satisfies the small cancellation condition $C(n)$ if no element in R is a product of fewer than n pieces.

The dependency on R being symmetrized does not present an issue. suppose we have $G = \langle S|R \rangle$ with R not being symmetrized. Then by taking $\bar{R}$ as seen in 3.7, then $G = \langle S|\bar{R} \rangle$

Example

Take the fundamental group of a genus-2 surface, presentation $\langle a_1, b_1, a_2, b_2 | [a_1, b_1][a_2, b_2] \rangle$. The set of pieces is the set of words in S with length 1, thus satisfying $C'(1/7)$ and $C(8)$.

These condition are obviously very similar with $C'(\lambda)$ implying $C(\frac{1}{\lambda} + 1)$. Sometimes, especially in older texts, we see groups with presentations satisfying $C'(1/6)$ referred to as sixth-groups, $C'(1/7)$ as seventh-groups etc. We shall refer to a group with a presentation satisfying $C'(\lambda)$ or $C(n)$ conditions as a $C'(\lambda)$ *group* or a $C(n)$ *group* respectively.

Small cancellation conditions have many variants, generalizations, and expansions into topics in both geometry and group theory. We concentrate our efforts on the work connecting small cancellation theory and hyperbolic groups, although other important and interesting applications are seen in small cancellation theory for special cube complex, introduced by Wise, and very small cancellation [18]. Depending on the λ , if a presentation satisfies $C'(\lambda)$, then there is a large list of properties that can be said about the group. For groups with presentations satisfying $C'(1/6)$ this is exceptionally true, and where we concern ourselves due to connections with hyperbolic groups. A main result regarding the metric small cancellation condition considers $C'(1/6)$ presentations and will be an important lemma in the succeeding subsection.

Lemma 3.10. Greendlinger's lemma. *Let G be a group with finite presentation $\langle S|R \rangle$ satisfying $C'(1/6)$. For every non-trivial word w in S such that $w =_G 1$ there is a subword u of w and $r \in R$ such that u is also a subword of r and $|u|_s > (1 - 3\lambda)|r|_s$*

A proof can be seen in [22].

3.3 Dehn's Algorithm

Dehn's algorithm is a very direct approach aiming to solve the word problem. It relies on all the information required being contained within the presentation of a group [7, chapter III.F].

Definition 3.11. *Dehn's Algorithm.* Let G be a group with presentation $\langle S|R \rangle$ such that R is cyclically reduced. Given a reduced word $w = w_0$ in S perform the following:

1. If $w = 1$, the empty word, then stop the procedure, concluding that $w_0 =_G 1$
2. If w contains a subword u such that $r = uv \in R$ and $\frac{1}{2}|r|_s < |u|_s$, replace u with v^{-1} in w . If no such subword exists, stop the procedure, concluding that $w_0 \neq_G 1$.
3. Reduce w and remove any subword $r \in R$ from w if possible.
4. Repeat the steps.

With our start word as w_0 , if we denote the word produced from every repeat of step 3 as w_{i+1} then it is clear to see that $|w_0| > |w_1| > |w_2| \dots$. It follows that the algorithm terminates in at most $|w|_s$ steps. For the algorithm to work, a presentation must have appropriate subwords u and v of relations.

If a group has a presentation that admits Dehn's algorithm to solve the word problem then the presentation is called a *Dehn Presentation*, formally:

Definition 3.12. *Dehn Presentation.* Let G be a group with finite presentation $\langle S|R \rangle$. If for all w , such that $w =_G 1$, there exists $r = uv \in R$ with $\frac{1}{2}|r|_s < |u|_s$ and $w = w_1uw_2$, then $\langle S|R \rangle$ is a Dehn presentation.

Originally Dehn proved that the fundamental groups of closed orientable two-dimensional manifolds admit the algorithm, thus were word and conjugacy problem solvable, remarked in chapter 5 of [27]. This was developed and expanded, leading us to the main theorem for this section.

Theorem 3.13. *Let G be a group with a presentation satisfying small cancellation condition $C'(1/6)$, then G is hyperbolic.*

To prove this we must first bring in equally exciting and important theorems regarding the groups with Dehn presentations.

Theorem 3.14. *Let G be a group with a finite presentation, $\langle S|R \rangle$, satisfying small cancellation condition $C'(1/6)$, then $\langle S|R \rangle$ is a Dehn presentation.*

Proof. The proof for this lemma follows from 3.10, if we have a word w in S such that $w =_G 1$, then there exists a sub-word u of w such that $(1 - 3(\frac{1}{6})|r|_s = \frac{1}{2}|r|_s < |u|_s$. This is exactly the definition of a Dehn Presentation. $\square$

To continue with the next theorem we introduce the Dehn function.

Suppose we have $G = \langle S | R \rangle$ with R symmetrized. From the definition of a presentation, and discussion regarding the reduction of words, we can see that a word w in S satisfies $w =_G 1$ if and only if

$$w =_{F(S)} \prod_{i=1}^n u_i r_i u_i^{-1}$$

where u_i is a word in S and $r_i \in R$, equivalently w must be the product of conjugates of relators. The definition of the area of a word follows from this.

Definition 3.15. *Area of a word.* Let $G = \langle S | R \rangle$ with R symmetrized and w in S such that $w =_G 1$. Then

$$Area(w) = \min\{n \mid w =_{F(S)} \prod_{i=1}^n u_i r_i u_i^{-1}\}$$

the fewest conjugates of relators needed to construct w .

If we aim to describe a relationship between the area of word and its word length we result in the *Dehn function*.

Definition 3.16. *Dehn function.* With w constructed as in 3.15, the Dehn function $\delta : \mathbb{N} \rightarrow \mathbb{N}$ is

$$\delta(n) = \max\{area(w) : |w|_s \leq n\}.$$

We will also require a theorem regarding Dehn's Function and hyperbolic groups, a lengthy and difficult proof for which can be found in [22].

Theorem 3.17. $G = \langle S | R \rangle$ with R symmetrized. Suppose there is a linear function $f(n)$, then G is hyperbolic if and only if $\delta(n) \leq f(n)$ for all n .

Theorem 3.18. Let G be a group with a Dehn presentation. Then G is a hyperbolic group.

Proof. By definition, a group with a Dehn presentation admits the Dehn algorithm. We have already seen that in this case for any word $w =_G 1$ the algorithm will terminate in at most $|w|_s$ steps, as at each step w can be shortened in word length by at least 1. The maximum area for a word of length n , is therefore equal to n . By 3.17 G is hyperbolic. $\square$

The implication actually works the other way as well [15], but as the intention is to start with a small cancellation group this is not a necessary fact.

It follows from 3.14 and 3.18 that 3.13 holds. Not only do $C'(1/6)$ groups exhibit δ -hyperbolicity, they are also word solvable. Typically to prove a word problem is solvable we only need to show that the Dehn function is bound by some calculable function, a much weaker property than the linear relation we see for $C'(1/6)$ groups [21].

Although this result is impressive, it does seem intuitive. We know that hyperbolic groups exhibit properties close to trees, hyperbolicity acting as a measure of similarity with the free groups. With a free group having presentation with no relations, it would make sense that by limiting how relations and cancellations behave, that we result in a group with properties similar to the free group. The link between exactly how much limitation is required before the group presented is hyperbolic, is quite a natural one.

This concludes and solves the problem we had at the end of section 2. The issue was that although we could define a hyperbolic group, actually producing groups we know to be hyperbolic was difficult. Without a method to derive a hyperbolic group we would have to start with an arbitrary group and attempt to determine if its Cayley graph exhibited δ -slim triangles. Small cancellation theory gives us an easy way to produce a group, via presentation, we know is hyperbolic. With this we have an abundance of groups that are easy to form with properties we desire. An added benefit being that they are both word and conjugacy problem solvable, the latter, more complicated, conclusion shown in chapter III.F of [7]. With this we can continue in our efforts and investigate the boundary of a hyperbolic group.

4 Methods

Now that we have a consistent method of producing hyperbolic groups, and an understanding of the theory and motivations behind them, we can develop a method for estimating and plotting the boundary of hyperbolic groups. We begin with a presentation for a small cancellation group, and this section will discuss the methods I have programmed to produce a plot estimating its boundary. The method is split between two languages; GAP to solve problems regarding groups, and Python to produce and display the plots. Any preexisting techniques used will be discussed, namely the Knuth-Bendix completion algorithm, implemented in GAP, and multidimensional scaling (*MDS*). This is done alongside discussing techniques developed by myself and the rationale and ideologies behind the methods.

GAP is a system for computational algebra, providing a programming language and a large list of packages written in this GAP language [1]. With applications in algebra, group theory and many more it will handle appli-

cation of the group theory for the method developed. A fundamental part of the system for our purposes being the *'kbmag'* package, developed and maintained by Derek Holt, which facilitates the running of the Knuth-Bendix completion algorithm on a presentation or group. If successful, producing a set of rules solving the conjugacy and word problems for the group [2] and providing many functions that are pivotal for handling words. Although other applications of the Knuth-Bendix algorithm exist, none discovered have the support and back-bone in other mathematical application that GAP supplies. The techniques and code produced were all done in GAP 4.11.1, ran in Linux through Windows Subsystem for Linux (WSL) 2. Ran as such due to both unfamiliarity with Linux but also for easier collaboration with Python. The version of kbmag used is 1.5.9.

Python is a much more common programming language with over 2 million questions asked on *stackoverflow.com* tagged 'python', compared to 'gap-system' with only 57. Python has many applications, however we're interested with its abundance of packages supporting machine learning. *scikit-learn* (or *sklearn*) is a very common Python package, capable of applying many machine learning techniques, analysis, and much more. Using scikit-learn we apply MDS, a technique to represent data in low dimensions, chosen due its robustness and thorough documentation.

Summarised, my method is:

1. Produce a presentation satisfying $C'(1/6)$, hence a hyperbolic group.
2. Get the Knuth-Bendix algorithm to run successfully on the presentation.
3. Generate a set of words $\{w_1, w_2, \dots, w_n\}$ representing elements on the sphere of a fixed point (the identity for ease).
4. Create an $n \times n$ dissimilarity matrix D such that $D_{i,j} = e^{-\epsilon(w_i, w_j)}$ for some $\epsilon > 0$.
5. Use MDS to transform D into Euclidean space of n dimensions (typically 2).
6. Plot the estimate of the boundary for the hyperbolic group.

Steps 1-4 are done in GAP while 5-6 are done in Python.

Any functions I developed that are referenced can be found in the Appendices. Further developed code, including examples and further functionality can be found at [33]. Documentation and further information regarding kbmag and sklearn can be found at [2] and [28] respectively.

4.1 The Knuth-Bendix Completion Algorithm

As we have already discussed and defined small cancellation groups we will begin with the assumption that we already have a presentation satisfying $C'(1/6)$. It is very possible to construct such presentations by hand and tools exist to ensure that they satisfy the required small cancellation condition. For example you may use the GAP package 'smallcancellation' found at <https://github.com/isadofschis/smallcancellation>. With this assumption we will discuss what the Knuth-Bendix algorithm is, how and why it is used and the benefits and drawbacks associated with it. The definitions follow from [11] and [23], though are briefly summarised and I would recommend reading [11].

It is worth mentioning that even though $C'(1/6)$ small cancellation groups were the main consideration, the functions and methods developed can be applied to any group constructable in GAP.

As we want to estimate the boundary of a group, it follows that we need a set of elements far away from a fixed point in the groups Cayley graph. For ease we will set this point to the identity, as the choice is arbitrary. It is not difficult to generate words, say by randomly concatenating a large number of elements in the generating set. The issue we have is that this word may represent an element that in turn is represented by a smaller word, the element is then closer to the identity than intended. Suitability for the boundary estimation is not given, even for lengthy words. Ideally we would like a method that gives us the *normal form* of the group, a way to produce the shortest word representing any element in the group. With this we could choose words and elements that we guarantee have a set distance from the identity, according to the word metric.

A rewriting system is a set of rules for replacing strings (or terms in general), typically such that equality remains. A rule $l \rightarrow r$ dictates that $ulv \rightarrow urv$ for all u, v and is called a rewrite rule. Similar to the steps we see in Dehn's algorithm, but formalised into a full set of $u \rightarrow v^{-1}$ rules. Though it is not required in $l \rightarrow r$ that l is smaller than r , as in Dehn's algorithm. Given a rewriting system where every input has a finite set of rewrite rules such that no rules can be applied to the end result, the system is said to *terminate*.

Confluence of a rewriting system describes how many ways there are of applying rules to a given input. If for some u, v, w we have rewrite rules $u \rightarrow_* v$ and $u \rightarrow_* w$, then confluence dictates that there must exist some x such that $v \rightarrow_* x$ and $w \rightarrow_* x$. $\rightarrow_*$ denoting a finite combination of rewrite rules. In a confluent rewriting system, although there may be many rewrite rules that can be applied, all of them will terminate at the same point for a given input. If we have a rewrite system that is both terminating and confluent, an input will always be rewritten into its normal form. The connections here to Dehn's algorithm, presentations and words

are clear.

The *Knuth–Bendix completion algorithm* is then an algorithm which, from a set of relations, aims to produce a confluent terminating rewriting system, with rules retaining equality.

The algorithm can end in three results

- *convergence*. A finite set of rewrite rules is determined.
- *failure*. The algorithm stops.
- *Divergence*. A loop is entered and the algorithm will not stop.

Knuth-Bendix algorithm convergence means that for a presentation $\langle S|R \rangle$ of a group and a word in S , the words normal form can be calculated, i.e the shortest word representing the same element. Equally given an element g in the group, the shortest word representing g can be found, with many applications extending from this [2]. Unfortunately convergence is not guaranteed, even for $c'(1/6)$ groups, of which we know have Dehn presentation and are word solvable.

A parameter of the Knuth-Bendix algorithm is the reduction ordering of generators. Typically these are in a default order according to how they appear in $\langle S|R \rangle$. For examples if we provide $\langle a, b, c \rangle$ then the reduction order is $a < a^{-1} < b < b^{-1} < c < c^{-1}$. The first rules generated are based off of this ordering and thus the ordering impacts completion of the algorithm. Details of exactly why failure can occur is covered in [11] and [13]. An example from [13] showcases this; the presentation $\langle a_1, b_1, a_2, b_2 | [a_1, b_1][a_2, b_2] \rangle$ fails with the default reduction ordering. If we set the ordering as $a_1 < a_1^{-1} < a_2 < a_2^{-1} < b_1 < b_1^{-1} < b_2 < b_2^{-1}$ then the algorithm converges. It is not always obvious what ordering, if any, is required.

Fortunately progress has been made in improving the knuth-bendix algorithm. One such example is the automatic group program, developed by Derek Holt, which is included as a method in the kbmag package [2]. This algorithm includes the Knuth-Bendix as a main component but has a different approach [2]. There are many cases where the Knuth-Bendix algorithm fails but the automatic group program succeeds [2]. $\langle a_1, b_1, a_2, b_2 | [a_1, b_1][a_2, b_2] \rangle$ with the default reduction ordering is an example of this. Unlike the Knuth-Bendix algorithm however, this program is only capable of being ran for groups, not monoids nor semi-groups. Not a problem for our means however, the main problem with this method is its slowness, it is preferable to exhaust orderings for the Knuth-Bendix before trying the automatic group program. Convergence is still not guaranteed either, presenting a considerable road block. If neither algorithm completes successfully then my method can not progress. The reader is referred to [20] for more information regarding the automatic group program.

With the successful running of either the Knuth-Bendix algorithm or automatic group program, we can now present elements in a group as words in normal form. Given a word we can find its normal form for the group. Thus we can continue with generating points to mimic the boundary.

4.2 Word Generation

Ideally generating words would be a simple endeavour, afterall the `kbmag` package includes a function `EnumerateReducedWords` that returns a list of elements with specified word length. Run this function and select the desired from the list. Regrettably, this just is not feasible. The growth function 2.7 of hyperbolic groups is too fast, especially considering we need points that are far away. Suppose we wanted to generate elements with length 1000, from a hyperbolic group with generating set $\{a, b\}$. We will take the growth function from $F(\{a, b\})$, although this will act as an upper bound, due to the properties of hyperbolic groups it will also act as an estimate. For $F(\{a, b\})$, there are $4 * 3^{1000-1} \approx 1.76 \times 10^{477}$ elements of word length 1000, requiring 1.4×10^{361} universes worth of storage [26] (taking each letter in a word as 8 bits). Another method is necessary.

We will therefore have to construct the words, fit for our needs. An old issue still prevails, it is easy to generate a random word and from there find the smallest word that represents the same element. It is not easy however to determine what the length of the element is before we construct the word. My method to generate n elements of word length 1 is then as follows:

1. Generate a random word longer than 1 .
2. Find the equivalent word of shortest length representing the same element.
3. Add the subword of 1 initial elements of the new shorter word to a list. If the new word is too short, disregard it.
4. If the list has n entries then stop, otherwise repeat.

Its easy to see that if a word is rewritten to be as short as possible, then any of its subwords can also not be shortened further. So the third step does not cause issues. Also due to the small cancellation conditions in place, it is difficult for the second step to reduce a words length enough that is disregarded.

This method is very fast. It is simple to generate a random word, regardless of length so starting with a longer word is not of much issue. Rarely are more than n iterations required, a stoppage of $5n$ total iterations has been placed however. Duplicate elements are also removed from the list to account for two of the random words representing the same element. This is taken into consideration when stopping. One should only expect a return

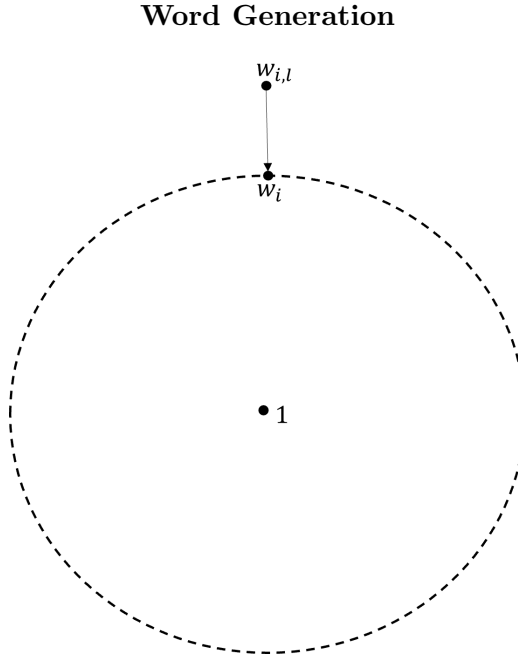


Figure 3: An illustration of the word generation method for a single element.

of less than n words when there are considerably less elements on the sphere of 1 than on the equivalent sphere on the free group of as many generators. Previous discussion shows this will likely never be the case. With this method we can produce a list of elements with set word length from the identity, a set to mimic the boundary.

Not in the scope of the project, but may be of interest to the reader, another word generation function was developed. Very similar to the above, but the words produced in step 1 have length 1 and are always accepted after step 2. This does not have application in the boundary but can be applied with techniques discussed later to display and evaluate the cancellation of a group and to plot where this occurs.

4.3 Dissimilarity Matrix

With a set of appropriate words, we now work towards gathering enough information about them to plot. A distance matrix is a symmetric matrix with pairwise distances of elements, although not necessarily from a metric. The ij^{th} elements being equal to the distance between the i^{th} and j^{th} elements of the set respectively. A matrix M is called a dissimilarity matrix if it is symmetric with $M_{i,i} = 0$ and $M_{i,j} \geq 0$ for all i, j [3]. They are very similar in nature but a dissimilarity specifies reflectivity and positivity [3], something we will require in the next step.

Recall that $d_{\epsilon,0}(\xi, \eta) = \inf\{\sum_{i=1}^n p_{\epsilon,o}(\xi_{i-1}, \xi_i)\}$ is a visual (pseudo)metric

on the boundary under certain parameters for ϵ , constructed as such to mimic the Euclidean metric. Our aim is to plot the words generated prior so we wish to imitate this metric as much as possible. Requiring the use of chains and defined on the boundary however, it is not suitable for the points we have. When extended to the boundary, the Gromov product retains properties it already had on elements in the space, it is suitable then for us to mimic the visual boundary for our points and define the distances between our points as $e^{-\epsilon(x,y)_1}$, dropping the summation and infimum. Thus, for a set of words $\{w_1, w_2, \dots, w_n\}$ we produce our dissimilarity matrix D such that

$$D_{i,j} = e^{-\epsilon(w_i, w_j)_1}$$

for all i, j . To best emulate the visual metric on the boundary, we wish to put ϵ under the same condition as seen in 2.33, although it is not often easy to determine δ . There are two issues taking this approach.

First, extending the Gromov product to the boundary resulted in corollary 2.31. In the finite space, the Gromov product can never be infinite, and $e^{-\epsilon(x,y)_1}$ can never equal 0. $D_{i,i} = e^{-\epsilon|w_i|_s}$, not 0 as required for a dissimilarity matrix. Remediating this, the function producing the matrix will manually check if $x = y$, in this case returning 0. To justify this we consider the points we generate as points on geodesic rays emanating from the identity (our base point). If two points differ then the rays have diverged and it is a safe assumption that they go on to define unique elements in the boundary. If the points are the same then we consider them on the same geodesic ray, taking the Gromov product as such.

The second issue is that using $e^{-\epsilon(x,y)_1}$ does not necessarily obey the triangle inequality, this fact causing the original extension to $d_{\epsilon,0}$. Our method does account somewhat for this, discussed further in the proceeding subsection and displayed in the *Results and Experimentation* section. In general though this remains a limitation of our method.

4.4 Multidimensional Scaling

We begin this subsection with a dissimilarity matrix, with the aim of plotting the points that generated it. Our data is almost certainly not Euclidean however, despite efforts to construct a metric with this consideration. To continue a method is required to translate the data into 2 or 3 dimensional Euclidean space. Our choice of method is multidimensional scaling or *MDS*. The following definitions and discussion mostly follow from [5]. MDS was, in part, chosen due its extensive use, popularity, and documentation, [5] having 8000+ google scholar citations.

There are a handful of variations of MDS but we concern ourselves with absolute multidimensional scaling, and non-metric multidimensional scaling. The package we are using, *sklearn.manifold.MDS*, applies only these two, however they should cover all of our needs [28].

Given a dissimilarity matrix D , generated from objects $\{w_1, w_2, \dots, w_n\}$, MDS aims to produce a set $\{y_1, y_2, \dots, y_n\} \in \mathbb{R}^n$ (n specified) such that $f(D_{i,j}) \approx d(y_i, y_j)$, as closely as possible. Here d is the Euclidean metric. Although the space need not be Euclidean, this is typically the case [5, Chapter 3] and where we are concerned. The different forms of approximation and functions giving rise to the different types of MDS. Typically a random starting set of points is chosen, then by minimising a loss function called *stress*, you obtain the set of points $\{y_1, y_2, \dots, y_n\}$ in your desired space that best maintains dissimilarities from the input. To obtain stress (also sometimes called strain) we define an error term as

$$\epsilon_{i,j}^2 = (f(D_{i,j}) - d(y_i, y_j))^2.$$

Summing the error for all pairwise distances and normalising we produce the stress function

$$\text{stress}_f\{w_1, w_2, \dots, w_n\} = \sqrt{\frac{\sum (f(D_{i,j}) - d(y_i, y_j))^2}{\sum d(y_i, y_j)^2}}.$$

The exact nature of this function, and how to minimise it however does depend on the exact method of MDS chosen.

sklearn uses an MDS method called absolute MDS, where it is assumed that the distances supplied are already absolute values in the Euclidean space, f is the identity. $D_{i,j}$ should correspond exactly to the distance between the i^{th} and j^{th} points generated in Euclidean space. A simple application of this method is shown in figure 4. [5, chapter 9] states that this method is irrelevant in an applications point of view. It is ideal here however, we have precisely curated our dissimilarity matrix to try and represent Euclidean space. Using absolute metric MDS we will generate a set of Euclidean points that represent the distances as best as possible without any transformation. Representing the boundary in Euclidean space with little distortion.

Despite the efforts made, our data may not be even close to Euclidean, a lot of time not even obeying the triangle inequality. Given how we are mapping exactly as if it was, the further from being Euclidean the data is, the poorer the output from absolute MDS will be. The dissimilarity matrix will always be accepted however and a graph produced, regardless of how poor it is, the stress will just be remarkably high. This is where non-metric MDS applies. The goal is exactly the same as standard MDS but focuses on the rank or order of the data, a less stringent application. Instead of enforcing that the distances remain equal, non-metric aims to hold that if $D_{i,j} < D_{k,l}$ in the input, then $d(y_i, y_j) < d(y_k, y_l)$ for the respective points in the embedded space [28].

Violations of the triangle inequality are not an issue for non-metric MDS [5, Chapter 2], but absolute MDS will become less and less appropriate the

Embedding a 3-D sphere into 2-D

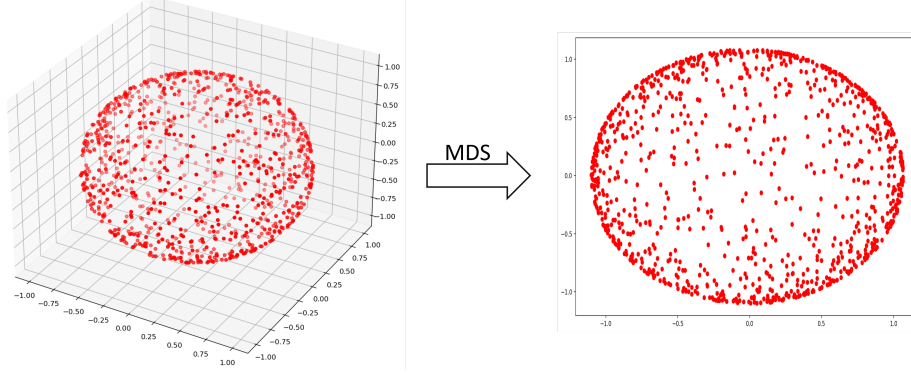


Figure 4: Embedding a sphere of radius 1 from 3-D to 2-D with absolute MDS.

further D gets from observing metric data. As both methods are supplied, we can switch between the two and hopefully find a suitable mapping into Euclidean space, regardless of the quality of the matrix. To aid this, alongside functions for transforming and plotting the data, a function has been supplied that returns how many triangles from the matrix D violate the triangle inequality. Noting that for n elements there are $\binom{n}{3}$ unique triangles, compare this to the total number of violations to contextualise severity.

MDS is far from the only technique that will map the points into Euclidean space, indeed there are multiple different MDS that will do so, each in a unique way. MDS was chosen due its popularity, thus documentation and accessibility, and acceptance of dissimilarity matrix. Further to this *sklearn* MDS has two techniques, theoretically covering all of our requirements. UMAP and tSNE are examples of other methods which will embed the dissimilarity matrix to Euclidean space and success may be possible with these also.

Having generated points representing the dissimilarity matrix in an appropriate space, we can easily generate a plot. Given a $C''(1/6)$ small cancellation group, I have created a set of functions that will plot an estimate for the boundary of the group.

5 Results and Experimentation

I have developed a method, and a set of functions to apply the method. This section aims to discuss the behaviour of said functions. It will go into details of practises found to produce the best results, as well as how functions behaviours are altered by changing parameters. This will begin with a plot of the free group, explaining the output seen. Discussion of

the parameter's of the main functions (stated below) will follow. These two functions combine many smaller steps and when combined will perform our goal in entirety.

The first function will return a dissimilarity matrix D as described in the *Dissimilarity Matrix* subsection. This is ran in GAP and performs steps 2-4 outlined in section *Methods*, it can be found in appendix A. For exploration and experimentation purposes, auxiliary functions have been produced that split up this main function. These also give few alternative avenues of producing the dissimilarity matrix, capable hopefully covering the majority of cases.

`GetVisualMatrix(g, n, l, eps)`

- `g` is the input of the group, as described in Chapter 47 of the GAP system manual.
- `n` is the number of unique elements to generate from `g` such that D will have size $n \times n$.
- `l` is the distance of the elements generated from the identity.
- `eps` is the value of ϵ used in $D_{i,j} = e^{-\epsilon(w_i, w_j)_1}$.

The second function discussed is the primary Python function. It will accept the matrix produced from `GetVisualMatrix`, perform MDS to estimate Euclidean points, returning them as Cartesian co-ordinates in an array and plotting these points. It can be found in appendix B. The MDS method itself has many parameters, of which are fixed when running my algorithm, the first subsection will discuss what these parameters are and why they were chosen. Translating data from GAP to Python is not trivial, especially with the former requiring to be ran in Linux, the latter primarily ran in windows. WSL2 is recommended for easier translation, with functions to read and write a CSV between the two available at [33], as well as auxiliary functions mentioned.

`plot_from_matrix(D, metric, title, dimensions)`

- `D` is the dissimilarity matrix returned as a *numpy* array.
- `metric` is a boolean argument, if `True` absolute MDS will be used, if `False`, non-metric MDS.
- `title` is an optional argument for the title of the output plot.
- `dimensions` is number of dimensions that the dissimilarity matrix is mapped to. An array of these points is returned and a plot produced if 2 or 3 is chosen.

Free group of 2 generators

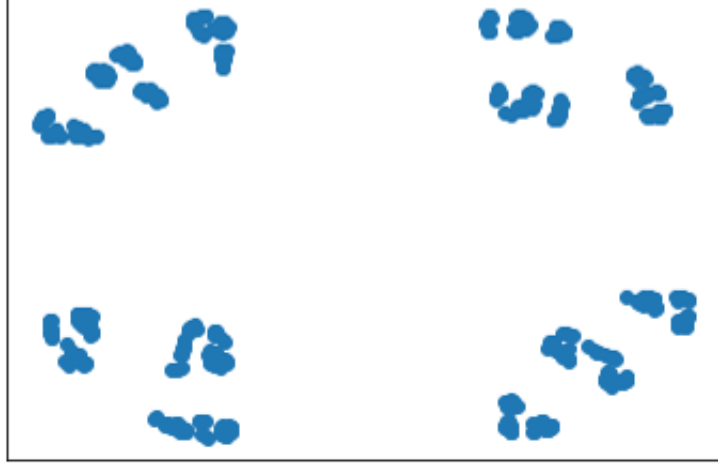


Figure 5: Plot estimating the boundary of the Free Group $F(\{a, b\})$.

5.1 The Free Group

To begin we will start with the simplest group. We will take the Free group $F(S)$ with $S = \{a, b\}$, aiming to simply produce a plot. There is no cancellation regarding pieces so choosing an appropriate distance is not too important as long as we choose a large sphere. The free group is also $H_{rips}(0)$ so choosing ϵ satisfying the condition of 2.33 is not possible. For `GetVisualMatrix` we set `n` as 500, `l` to 300, and `eps` to 0.5. For `plot_from_matrix` we insert the matrix produced as `D`, this matrix observes the triangle inequality so we decide `metric = True` and choose 2 dimensions. The resulting estimation displayed as figure 5.

The plot may look different than expected, especially as there are 500 points but all are in areas of high density. There are a clear 4 main clusters, each of these split up into 3, with some smaller separation within these. The 4 main clusters each correspond to one of the directions a, a^{-1}, b, b^{-1} . The reason we see these split so evenly is that the generation of words is random and there is little, to no, cancellation. Suppose the first letter of a word is a , then for any other word there is a 75% chance that it starts with another letter. In this case the identity lies on the geodesic between the two words, the Gromov product is 0 and our imbued visual metric is 1. If the words start with the same letter, then there are 3 options for the second letter, causing the second split seen in clusters. The Gromov product in this case being 1 and the visual metric 0.61. In general the geodesic between two points is more likely to travel closer to, if not through, the identity than not. These properties are seen in the snippet of D , figure 6. As all hyperbolic groups share similar properties to the free group, the dissimilarity matrix

tends to look as in 6 for all small cancellation groups. The plots tending to display clusters representing the number of generators.

Dissimilarity matrix example

	w_1	w_2	w_3	w_4	w_5	w_6	w_7	w_8
w_1	0.00	0.61	1.00	1.00	1.00	1.00	1.00	0.61
w_2	0.61	0.00	1.00	1.00	1.00	1.00	1.00	0.61
w_3	1.00	1.00	0.00	1.00	1.00	0.37	0.61	1.00
w_4	1.00	1.00	1.00	0.00	0.61	1.00	1.00	1.00
w_5	1.00	1.00	1.00	0.61	0.00	1.00	1.00	1.00
w_6	1.00	1.00	0.37	1.00	1.00	0.00	0.61	1.00
w_7	1.00	1.00	0.61	1.00	1.00	0.61	0.00	1.00
w_8	0.61	0.61	1.00	1.00	1.00	1.00	1.00	0.00

Figure 6: Dissimilarity matrix of 8 points from the Free Group of 2 generators.

5.2 Input Parameters

In order to control inputs and measure affects within parameters, a fixed group is chosen. In this case the fundamental group of a genus-2 surface, $\langle a_1, b_1, a_2, b_2 | [a_1, b_1][a_2, b_2] \rangle$. The boundary of this group is known to be homeomorphic to a circle, thus we have a baseline for our output.

To began we will discuss the parameters of `plot_from_matrix`. As the matrix input is produced from GAP, it will not be discussed here. The only other input then that may have unpredictable changes to the output is the `metric` argument. This argument affects the parameters chosen for the embedded MDS inside `plot_from_matrix`. The functions applying MDS themselves have many parameters although these have been fixed, dependant on choice of `metric`. These fixed parameters, and reasoning behind the choices are:

- `n_init` controls how many times the MDS algorithm will be ran, the result with the least stress taken. This is important as the algorithm begins with a random set of points, multiple runs help diminish the issue of getting 'stuck' in a local minima [5, Chapter 13]. In my function this is always set to 5, enough to mitigate the issue in my testing, without adding unnecessary run time.
- `max_iter` controls how many iterations the algorithm will perform for each run. This is set at 10,000 for `metric = True` and 100,000 for `False`. Chosen to maintain a roughly 10 seconds run time for both options.
- Remaining parameters are the defaults supplied. Changing of these was tested and did not affect output considerably if at all. `n_jobs`

allows for using multiple cores, ideally set to 5 to perform each run in parallel, this was left at 1 to not limit execution to certain systems.

To compare between the different options for `metric`, 2 matrices were generated with `GetVisualMatrix`. Both with fixed `l= 300` and `n= 500`, changing only `eps` with values 0.25 and 4. One matrix therefore obeying the triangle inequality whilst the other has data far from being metric, allowing the testing of the different methods of MDS, as discussed in subsection *Multidimensional scaling*. Running these on both non-metric and absolute MDS produces plots seen in figure 7. The top left plot showcases behaviour very similar to the free group, although with 8 clusters due to having more generators. The absolute MDS plot with $\epsilon = 4$ is similar to the other absolute MDS plot but the clusters a lot more condensed. With ϵ so high anytime the Gromov product is not 0, the visual metric is very small, forcing the clusters together. The most obvious observation is the failure of non-metric MDS. Despite the dissimilarity matrix for $\epsilon = 4$ heavily violating the triangle inequality, non-metric MDS is still weaker than absolute MDS. The reason for this is shown in figure 6; $D_{i,j} = 1$ the majority of the time. This is not compatible with the ordering method of non-metric MDS. With so many cases where $D_{i,j} = D_{k,l}$, there are not enough enforceable orderings to properly generate Euclidean points. Resulting in what appears a random assortment. My recommendation is that ϵ is chosen to be small and used with absolute MDS, rather than choose ϵ large with the plan to use non-metric MDS.

We now move to the parameters for `GetVisualMatrix`. `n` is the easiest of these to decide, not much exploration is required. You certainly don't want too few, a minimum of 100. Due to the way the points cluster, there does not tend to be an upper limit in terms of visual output, computation and time becoming the main concern, the recommendation is a max of 1000. All the plots seen in this section are from 500 generated points and it is rare that this number is not appropriate. I'm yet to have major concern with any outputs, nor computation times with `n` set to 500.

The next easiest parameter to explore is `eps`. We have already deduced the behaviour of `eps` (ϵ) on the output. A higher value forcing elements to be closer, this is showcased further in figure 8. From discussion the clear choice is a lower `eps`, hopefully one that results in a dissimilarity matrix that will obey the triangle inequality. Although consideration has to be made regarding the number of generators. The number of clusters corresponds to the number of generators, so in cases where there are more generators its best to have a slightly larger `eps`. This 'tightens up' the points together and gives clearer clusters. Unfortunately given the properties of ϵ discussed in subsection *Gromov boundaries* and here, there's no best `eps`. It is recommended to start with 0.25 and adjust based on output.

Finally we move to the `l` parameter of `GetVisualMatrix`. Figures 7,

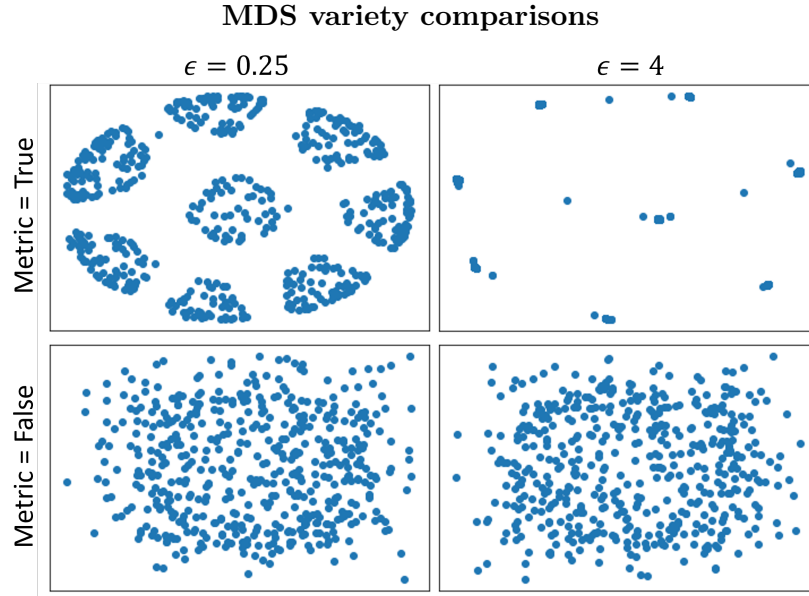


Figure 7: $\langle a_1, b_1, a_2, b_2 | [a_1, b_1][a_2, b_2] \rangle$ with differing `eps` (ϵ) and MDS method.

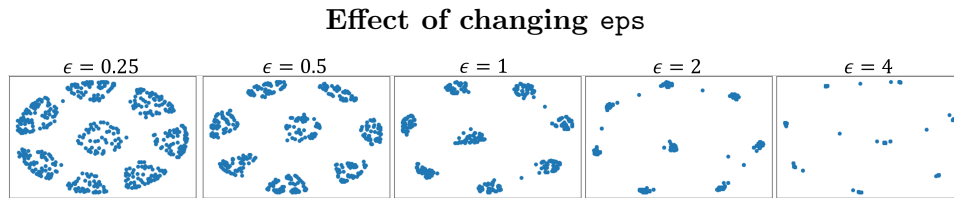


Figure 8: $\langle a_1, b_1, a_2, b_2 | [a_1, b_1][a_2, b_2] \rangle$ with multiple `eps` (ϵ).

8 and 5 were all done with 1 at 300, i.e all elements generated had word length of 300. Typically this is suitable but larger numbers will represent the boundary better, there is no upper bounds in terms of output quality, only computational requirements. Importantly 1 must be large enough to represent the cancellation that will occur, for example taking a sphere of radius 4, the fundamental group of the genus two surface will look exactly the same as the Free group of as many generators. For all groups it's necessary find a radius that allows for reduction opportunities, enough to mimic the boundary and to display the true shape of the group. It makes sense then that a longer radius is required for presentations with larger pieces and longer relations.

Note that it seems from figures 7 and 8 that some of the boundaries points are near the origin. This is simply a transformation issue due to the representation in 2-D. The function `plot_from_matrix` has the capability to plot in 3-D, interactively if desired, doing so shows a much cleaner mapping into Euclidean space and better represents the shape we'd expect from the group. Figure 9 relays this. It is sometimes difficult to present an abstract space in 2-D and distortion will occur, the same is true for 3-D although there is some alleviation.

Fundamental group of a genus-2 surface, 3-D

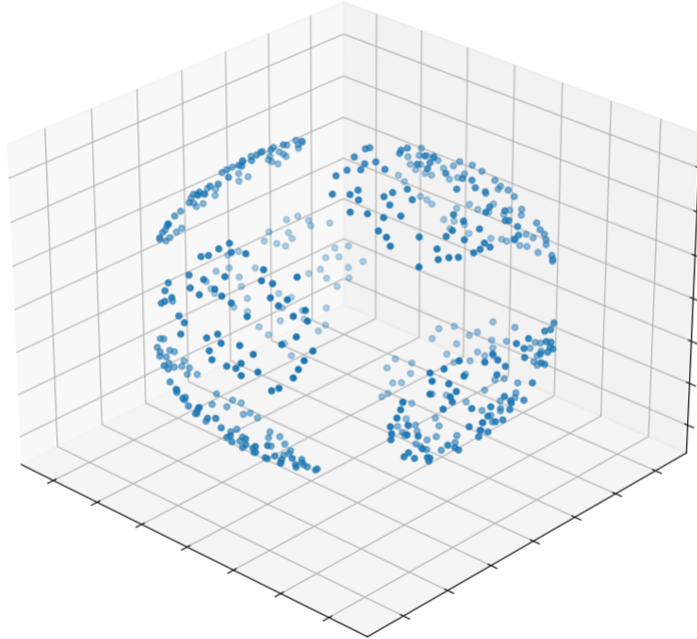


Figure 9: $\langle a_1, b_1, a_2, b_2 \mid [a_1, b_1][a_2, b_2] \rangle$ with $\epsilon = 0.25$ in 3-D.

5.3 Plotting an Estimation for the Boundary of $C'(1/6)$ Small Cancellation Groups

This subsection will apply the method to a small collection of groups. Testing the method and allowing for some displays of its capabilities. The discussion in subsection *The Free Group* will apply to most groups, the shape of these plots following the same logic due to the links of hyperbolicity and trees.

Unless stated otherwise, the following parameters were applied:

- `GetVisualMatrix`
 - `n = 500, l = 300, eps = 0.25`
- `plot_from_matrix`
 - `dimension = 2, metric = True`

taking into account requirements for a successful rewriting system.

To begin we start with $\langle a, b | abab^2ab^3 \dots ab^{20} \rangle$. The Knuth-Bendix algorithm (KBA) diverges for the reduction orderings, $a < a^{-1} < b < b^{-1}$, $b < b^{-1} < a < a^{-1}$, $a < b < a^{-1} < b^{-1}$ and $b < a < b^{-1} < a^{-1}$. Hence we move to attempting the automatic group program (AGP), which is also not successful. Without a confluent rewriting system we can not continue, a drawback of which a solution has not been found.

We move to example 7 from [36], $\langle a, b, c, d | a^2b^2c^2d^2 \rangle$. Similar to prior, an appropriate reduction ordering for the KBA could not be found, though the AGP was successful. From experimentation, the functions found in [2] for a confluent system are slower if that system is AGP rather than KBA, thus `n` was set to 250. The resulting dissimilarity matrix obeying the triangle inequality.

Take the presentation $\langle a, b, c, d | a^2b^2c^2da^{-2}b^{-2}c^{-2}, b^2d^2a^2cb^{-2}d^{-2}a^{-2} \rangle$. Similarly this fails Knuth-Bendix for attempted orderings, but the AGP works. `n` is set to 250, the triangle equality is also observed. The plots estimating these two hyperbolic groups, shown in figure 10 are very similar to each other, and the genus-2 surface; an expectation due to all having 4 generators.

As mentioned prior, the functions can be applied to any infinite group if one of the two rewriting systems from [2] converge. Although designed with hyperbolicity and small cancellation in mind, a brief detour into other groups may give us a better idea of the capabilities of the functions.

We begin with the infinite dihedral group, $\langle a, b | a^2, (ab)^2 \rangle$. KBA converges with default ordering. There are only 4 points on sphere of radius 300. Standard word generation approach was not applicable and `GetReducedWords2`, seen in appendix A, was used. The resulting 'Gromov boundary' estimation displayed in figure 11.

Further small cancellation groups

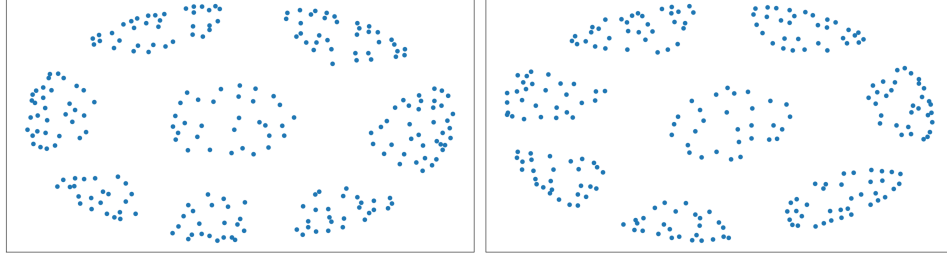


Figure 10: Gromov boundary estimations for $\langle a, b, c, d | a^2 b^2 c^2 d^2 \rangle$ (left) and $\langle a, b, c, d | a^2 b^2 c^2 d a^{-2} b^{-2} c^{-2}, b^2 d^2 a^2 c b^{-2} d^{-2} a^{-2} \rangle$ (right) .

Taking $\mathbb{Z}^3 = \langle a, b, c | [a, b], [a, c], [b, c] \rangle$ as another example. KBA converges with default ordering, although the parameters do not produce a dissimilarity matrix observing the triangle inequality. The results for this, seen in figures 11 and 12, may be different than expected. The 3-D plot especially so. The embedding from 3-D to 3-D is not particularly interesting itself, but the 'metric' has been altered between spaces with the graph representing this deformity.

Infinite dihedral group and $\mathbb{Z}^3$ in 2-D

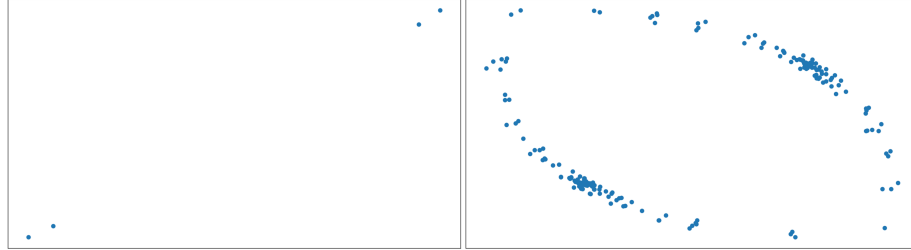


Figure 11: Boundary estimations for the infinite dihedral group $\langle a, b | a^2, (ab)^2 \rangle$ (left) and $\mathbb{Z}^3 = \langle a, b, c | [a, b], [a, c], [b, c] \rangle$ (right) .

$\mathbb{Z}^3$ in 3-D

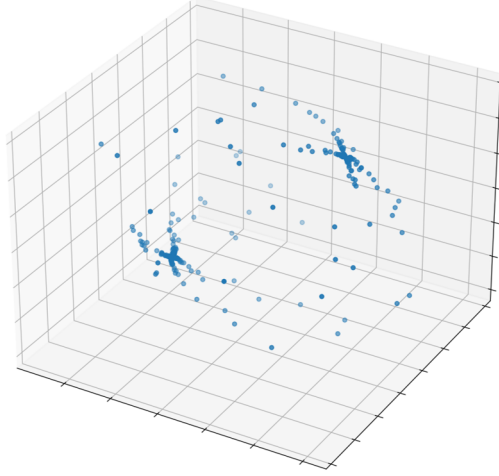


Figure 12: Boundary estimations for $\mathbb{Z}^3, \langle a, b, c | [a, b], [a, c], [b, c] \rangle$ in 3 dimensions.

References

- [1] Gap system. <https://www.gap-system.org/index.html>. Accessed: 2022-07-19.
- [2] Knuth-bendix on monoids and automatic groups. <https://docs.gap-system.org/pkg/kbmag/doc/chap0.html>. Accessed: 2022-07-19.
- [3] Henry Adams, Mark Blumstein, and Lara Kassab. Multidimensional scaling on metric measure spaces. *Rocky Mountain Journal of Mathematics*, 50(2):397–413, 2020.
- [4] Benjamin James Barrett. *Detecting topological properties of boundaries of hyperbolic groups*. PhD thesis, University of Cambridge, 2018.
- [5] Ingwer Borg and Patrick JF Groenen. *Modern multidimensional scaling: Theory and applications*. Springer Science & Business Media, 2005.
- [6] Brian H Bowditch. A course on geometric group theory. 2006.
- [7] Martin R Bridson and André Haefliger. *Metric spaces of non-positive curvature*, volume 319. Springer Science & Business Media, 2013.
- [8] Bruce Chandler and Wilhelm Magnus. *The Foundation: Dyck’s Group-Theoretical Studies*, pages 5–10. Springer New York, New York, NY, 1982.
- [9] Michel Coornaert and Athanase Papadopoulos. *Symbolic dynamics and hyperbolic groups*. Springer, 2006.
- [10] Tushar Das, David Simmons, and Mariusz Urbański. *Geometry and dynamics in Gromov hyperbolic metric spaces*, volume 218. American Mathematical Soc., 2017.
- [11] Alan JJ Dick. An introduction to knuth–bendix completion. *The Computer Journal*, 34(1):2–15, 1991.
- [12] Reinhard Diestel. The basics. In *Graph Theory*, pages 1–34. Springer, 2017.
- [13] David BA Epstein, Derek F. Holt, and SE Rees. The use of knuth–bendix methods to solve the wordproblem in automatic groups. *Journal of Symbolic Computation*, 12(4-5):397–414, 1991.
- [14] Trevor Evans. On multiplicative systems defined by generators and relations: I. normal form theorems. In *Mathematical Proceedings of the Cambridge Philosophical Society*, volume 47, pages 637–649. Cambridge University Press, 1951.

- [15] Jane Lopez de Gamiz. Hyperbolic groups. https://warwick.ac.uk/fac/sci/math/people/staff/lopezdegamiz/hyperbolic_groups.pdf, 2018.
- [16] Rostislav Grigorchuk and Igor Pak. Groups of intermediate growth: an introduction for beginners. <https://arxiv.org/abs/math/0607384>, 2006.
- [17] Mikhael Gromov. Hyperbolic groups. In *Essays in group theory*, pages 75–263. Springer, 1987.
- [18] Vincent Guirardel. Geometric small cancellation. In *IAS/PARK CITY MATHEMATICS SERIES*. Citeseer, 2012.
- [19] A Haefliger and E Ghys. Group theory from a geometrical viewpoint, 1991.
- [20] Derek F Holt. The warwick automatic groups software. In *Geometric and Computational Perspectives on Infinite Groups: Proceedings of a Joint DIMACS/Geometry Center Workshop, January 3-14 and March 17-20, 1994*, volume 25, page 69. American Mathematical Soc., 1996.
- [21] James Howie. Hyperbolic groups. <https://www.macs.hw.ac.uk/~jim/samos.pdf>, 2000. Accessed: 2022-08-05.
- [22] Michael Hull. Hyperbolic groups. <http://homepages.math.uic.edu/~mbhull/hyperbolic%20lecture%20notes.pdf>, 2015. Accessed: 2022-07-14.
- [23] Wolfram Research, Inc. Mathematica, Version 13.1. Champaign, IL, 2022.
- [24] David Lawrence Johnson and David Leroy Johnson. *Topics in the theory of group presentations*, volume 42. Cambridge University Press, 1980.
- [25] Ilya Kapovich and Nadia Benakli. Boundaries of hyperbolic groups. *arXiv preprint math/0202286*, 2002.
- [26] Seth Lloyd. Ultimate physical limits to computation. *Nature*, 406(6799):1047–1054, 2000.
- [27] Roger C Lyndon, Paul E Schupp, RC Lyndon, and PE Schupp. *Combinatorial group theory*, volume 188. Springer, 1977.
- [28] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and

- E. Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- [29] Derek JS Robinson. *A Course in the Theory of Groups*, volume 80. Springer Science & Business Media, 2012.
 - [30] Joseph J. Rotman. *The Word Problem*, pages 418–470. Springer New York, New York, NY, 1995.
 - [31] Viktor Schroeder. Quasi-metric and metric spaces. *Conformal Geometry and Dynamics of the American Mathematical Society*, 10(18):355–360, 2006.
 - [32] Nick Scott. Growth of finitely generated groups, 2007.
 - [33] Bryce Shakespeare. Estimating the Boundary of Hyperbolic Groups. <https://github.com/BryceShakes/Estimating-the-Boundary-of-Hyperbolic-Groups>.
 - [34] Hamish Short. Diagrams and groups. *The Geometry of the Word Problem for Finitely Generated Groups*, Birkhäuser, 2007.
 - [35] John Stillwell. *Classical topology and combinatorial group theory*, volume 72. Springer Science & Business Media, 1993.
 - [36] Ralph Strebel. Small cancellation groups. In *Sur les groupes hyperboliques d’après Mikhael Gromov*, pages 227–273. Birkhäuser Basel, 1990.

Appendix A GAP functions

Note that only functions are below. Commented code, along with examples and a tutorial can be found at [33] .

```
GetReducedWords := function(g, n, l, rws...)
  local words, kb_g, i, w;
  words := [];
  AssignGeneratorVariables(g);
  if Length(rws) = 0 then
    kb_g := KBMAGRewritingSystem(g);
    KnuthBendix(kb_g);
  else
    kb_g := rws[1];
  fi;
  for i in [1..n*5] do
    w := ReducedForm(kb_g,
      UnderlyingElement(
        PseudoRandom(g: radius := 1*5)));
    if Length(w) >= 1 then
      Add(words, Subword(w, 1, 1));
    fi;
    if Length(words) = n then
      words := DuplicateFreeList(words);
    fi;
    if Length(words) = n then
      break;
    fi;
  od;
  return DuplicateFreeList(words);
end;

GetReducedWords2 := function(f, n, l)
  local kb_f, rando, words, points, pos, i;
  rando := RandomSource(IsMersenneTwister, 42);
  if IsKBMAGRewritingSystemRep(f) then
    words := EnumerateReducedWords(f, l, l);
    points := [];
    for i in [1..n] do
      pos := Random(rando, 1, Length(words));
      Add(points, words[pos]);
      Remove(words, pos);
    od;
  fi;
end;
```

```

        if Length(words) = 0 then
            break;
        fi;
    od;
else
    kb_f:=KBMAGRewritingSystem(f);
    KnuthBendix(kb_f);;
    words:=EnumerateReducedWords(kb_f,1,1);
    points:=[];
    for i in [1..n] do
        pos := Random(rando,1,Length(words));
        Add(points, words[pos]);
        Remove(words, pos);
        if Length(words) = 0 then
            break;
        fi;
    od;
fi;
Unbind(words);
return points;;
end;

```

```

GetReducedFreeWords := function(g, n, l)
    local un_words, kb_g, red_words, j;;
    un_words:= GetFreeWords(g, n*2, l);;
    if IsKBMAGRewritingSystemRep(g) then
        kb_g := g;;
    else
        kb_g:=KBMAGRewritingSystem(g);;
        KnuthBendix(kb_g);;
    fi;;
    red_words:=[];;
    for j in un_words do
        Add(red_words, ReducedForm(kb_g, j));;
        if Length(red_words) = n then
            red_words:=DuplicateFreeList(red_words);;
        fi;
        if Length(red_words) = n then
            break;
        fi;
    od;
    return red_words;;
end;;

```

```

WordMetric := function(g, w1, w2)
    local kb_g;
    if IsKBMAGRewritingSystemRep(g) then
        return Length(ReducedForm(g, w1^-1*w2));;
    else
        kb_g:=KBMAGRewritingSystem(g);;
        KnuthBendix(kb_g);;
        return Length(
            ReducedForm(kb_g, w1^-1*w2));;
    fi;
end;;

GromovProduct := function(g, w1, w2, w3...)
    local kb_g, id;
    if Length(w3)=0 then
        id := w1^-1*w1;;
    else
        id := w3[1];;
    fi;
    if w1 = w2 then
        return 1.0/0.0;;
    else
        if IsKBMAGRewritingSystemRep(g) then
            return (WordMetric(g, id, w1) +
                WordMetric(g, id, w2) -
                WordMetric(g, w1, w2))*0.5;;
        else
            kb_g:=KBMAGRewritingSystem(g);;
            KnuthBendix(kb_g);;
            return (WordMetric(kb_g, id, w1) +
                WordMetric(kb_g, id, w2) -
                WordMetric(kb_g, w1, w2))*0.5;;
        fi;
    fi;
end;

VisualMetric := function(grom, eps)
    return Exp(Float(-eps*grom));;
end;;

```

```

VisualMatrix := function(g, words, eps)
  local matrix, i, j, row, gromo, vis, kb_g;
  matrix := [];
  if IsKBMAGRewritingSystemRep(g) then
    for i in words do
      row := [];
      for j in words do
        vis := VisualMetric(
          GromovProduct(g, i, j), eps);
        Add(row, vis);
      od;
      Add(matrix, row);
    od;
  else
    kb_g:=KBMAGRewritingSystem(g);
    KnuthBendix(kb_g);
    for i in words do
      row := [];
      for j in words do
        vis := VisualMetric(
          GromovProduct(g, i, j), eps);
        Add(row, vis);
      od;
      Add(matrix, row);
    od;
  fi;
  return matrix;
end;

GromovProductMatrix := function(g, words)
  local matrix, i, j, row, gromo, kb_g;
  matrix := [];
  if IsKBMAGRewritingSystemRep(g) then
    for i in words do
      row := [];
      for j in words do
        gromo := GromovProduct(g, i, j);
        Add(row, gromo);
      od;
      Add(matrix, row);
    od;
  else
    kb_g:=KBMAGRewritingSystem(g);

```

```

        KnuthBendix(kb_g);;
    for i in words do
        row :=[];;
        for j in words do
            gromo := GromovProduct(kb_g, i, j);;
            Add(row, gromo);;
        od;
        Add(matrix, row);;
    od;
fi;
return matrix;;
end;

```

```

WordMetricMatrix := function(g, words)
    local matrix, i, j, row, w_met, kb_g;
    matrix := [];
    if IsKBMAGRewritingSystemRep(g) then
        for i in words do
            row :=[];;
            for j in words do
                w_met := WordMetric(g, i, j);;
                Add(row, w_met);;
            od;
            Add(matrix, row);;
        od;
    else
        kb_g:=KBMAGRewritingSystem(g);;
        KnuthBendix(kb_g);;
        for i in words do
            row :=[];;
            for j in words do
                w_met := WordMetric(g, i, j);;
                Add(row, w_met);;
            od;
            Add(matrix, row);;
        od;
    fi;
    return matrix;;
end;

```

```

GetVisualMatrix := function(g, n, l, eps, rws...)
    local kb_g, words, i, w;

```



```

    AssignGeneratorVariables(g);;
    if Length(rws) = 0 then
        kb_g:=KBMAGRewritingSystem(g);;
        KnuthBendix(kb_g);;
    else
        kb_g := rws[1];;
    fi;
    words:=[];
    for i in [1..n*5] do
        w := ReducedForm(kb_g,
            UnderlyingElement(
                PseudoRandom(g: radius := 1*10)));;
        if Length(w) >= 1 then
            Add(words, Subword(w, 1, 1));;
        fi;
        if Length(words) = n then
            words:=DuplicateFreeList(words);;
        fi;
        if Length(words) = n then
            break;
        fi;
    od;
    words:=DuplicateFreeList(words);;
    return VisualMatrix(kb_g, words, eps);;
end;;

MatToRec := function(mat);
    return(rec(1 := mat));;
end;;

GetVisualMatrixRec := function(g, n, l, eps)
    return MatToRec(GetVisualMatrix(g, n, l, eps));;
end;;

PrintMatrix := function(mat, path)
    PrintCSV(path, [MatToRec(mat)]);;
end;;

```

Appendix B Python functions

```

def plot_from_matrix(D, metric=True,
                    title = None, dimensions = 2):
    if metric == False:
        mds = manifold.MDS(n_components=dimensions,
                           max_iter=100000,
                           n_init = 5,
                           eps=1e-3,
                           metric = False,
                           dissimilarity="precomputed")
        new = mds.fit(D).embedding_
    elif metric:
        mds = manifold.MDS(n_components=dimensions,
                           max_iter=10000,
                           n_init = 5,
                           eps=1e-3,
                           metric = True,
                           dissimilarity="precomputed")
        new = mds.fit(D).embedding_
    if dimensions == 2:
        plt.figure(figsize=(12.80,7.20))
        plt.scatter(new[:,0], new[:,1])
        if title != None:
            plt.title(title)
        plt.xticks([])
        plt.yticks([])
        plt.show()
    elif dimensions == 3:
        fig = plt.figure(figsize=(19.20,10.80))
        ax = fig.add_subplot(projection='3d')
        ax.scatter(new[:,0], new[:,1], new[:,2])
        ax.xaxis.set_ticklabels([])
        ax.yaxis.set_ticklabels([])
        ax.zaxis.set_ticklabels([])
        if title != None:
            plt.title(title)
        plt.show()
    return new

def csv_to_matrix(path):
    assert os.path.isfile(path)
    df = pd.read_csv(path)

```

```

if df.shape[1] > 1:
    arr = []
    for i in range(df.shape[1]):
        arr.append(eval(df.iloc[0,i]))
    arr = np.array(arr)
elif df.shape[1] == 1:
    arr = np.array(eval(df.iloc[0,0]))
return arr

def plot_from_csv(path, metric = True,
title = None, dimensions =2 ):
arr = csv_to_matrix(path)
return plot_from_matrix(arr, metric=metric,
title=title, dimensions = dimensions)

```